Scalar Self-Seeded Arithmetic

A new whole-domain presentation of iPRA, and the limits of the current checker reductions

Research prepared for Vladimir Reshetnikov with ChatGPT

10 November 2026

Abstract

We give a scalar recursion basis with neither an independent initializer nor an old-counter variable in its update. Its equations are $F(0; v.h) = S(h(0))$ and $F(Sn; v.h) = h(F(n; v.h))$. With a Boolean test of actual equality, successor injectivity, the single separation sentence $0 = S0 \to \bot$, literal-equation induction, and ordinary intuitionistic first-order equality logic, this theory is bi-interpretable and equiconsistent with full first-order iPRA. Numerical coverage is derived. The substantive construction uses a finite-case predecessor, bounded selectors, and two temporary initialization states; every recursive body is compiled back into the original source language.

The complexity objective remains unresolved. Independent combinations of existing checker ideas reduce the observed byte minimum from 558 to 554 and the observed AST minimum from 296 to 291, using different programs. Neither those small implementation gains nor the new arithmetic theorem establish a significant complete-checker breakthrough. No measured checker for the new arithmetic basis is supplied. We keep the arithmetic theorem, inherited kernel metatheory, implementation measurements, and finite tests explicitly distinct.

Contents

1 Results, benchmark, and precise status 3

2 All logical rules of the new arithmetic theory 4

3 Scalar self-seeded iteration with actual equality 4

4 A source compiler for bounded scalar iteration 8

5 Arithmetic graphs without a circular size bound 10

6 Full primitive recursion and both interpretations 12

7 The separate complete-checker result 14

8 Measurement, tests, and evidence 15

9 A discarded freshness shortcut and its exact failure 16

10 Further research questions 16

A Attribution and the complete inherited checker theory 18

B Using the initializer to select computation 18

C The complete checker and its interpretation 20

D Complementary introductions with one local assumption 24

E Arithmetic strength and both ordinary interpretation composites 29

F Internal scalar predecessor bootstrap 31

G From literal-equation induction to full intuitionistic PRA 35

H Pairing, primitive recursion, and internal numerical composites 40

1 Results, benchmark, and precise status

The target is full first-order intuitionistic primitive recursive arithmetic, with symbols indexed by fixed finite primitive-recursive descriptions, their defining equations, successor separation and injectivity, and quantifier-free induction. Its logical language includes genuine falsity and all ordinary first-order connectives and quantifiers. This is the first-order convention used in the current drafts and in the standard discussion of iPRA [12, 13]. It is stronger as an interpretation target than merely reproducing the same numerical functions or the same quantifier-free theorems.

Theorem 1.1 (Arithmetic result)**.** *The theory $T_{F\epsilon}$ specified in Sections 2 and 3 is bi-interpretable and equiconsistent with full first-order* iPRA*. Both interpretations use the entire numerical domain and actual equality. Each composite is internally isomorphic to the identity by $x = y$; all formula composites are provably equivalent.*

The proof occupies Sections 3–6. It is a written mathematical proof, with a separate same-session assistant review. It has not been formalized in Lean or Rocq. Numerical experiments are supplementary checks of formulas and implementations.

The primitive recursive constructor has the schematic shape

$$F(0; v.h) = S(h(0)), \qquad F(Sn; v.h) = h(F(n; v.h)).$$

Only the previous state v is bound in h. This differs from the supplied B_ϵ basis [2], whose update also sees the old counter. The price of the scalar restriction is an explicitly postulated successor-injectivity axiom; B_ϵ derives that property. Therefore the result is a new structural trade-off, not a proved domination of B_ϵ in total axiom or checker complexity. The independent-seed removal and use of actual equality already occur in B_ϵ; they are not claimed as new by themselves.

The current comparison is frozen at ProveIt commit `76a2651601d17b34d46753f189ea026a21ce8d29` [1]. All 17 direct archives were matched to the GitHub Git-blob SHA-1 and byte length before use. Recursive inspection found 32 distinct ZIP payloads and 443 distinct Python sources. All 38 sources on or below a candidate coordinate minimum were classified: three are complete checkers and 35 are supporting utilities. The corpus inventory, classifications, hashes, and all 443 source files are included.

Complete program	Norm. bytes	Tokens	AST	Gzip-9
Previous certified-zero byte profile	558	264	310	345
Previous certified-zero joint profile	598	262	296	350
New lexical byte composition	**554**	267	311	339
New lexical loop composition	642	263	**291**	353
New loop, separate input assignment	641	262	293	354
New explicit-list composition	562	267	309	344

The previous coordinate minima are 558/262/296; the family now reaches 554/262/291, again with different programs. No single new program improves all three coordinates. The 554-byte program improves bytes by four (about 0.72%), while its token and AST counts increase. The 291-node program improves AST count by five (about 1.69%), while its bytes and tokens increase. Gzip is informational and is not an optimization objective. These are achieved sizes, hence upper bounds on an unknown optimum; they are not a theorem giving a lower bound on every possible axiomatization.

What the package does not establish. It does not establish the requested significant reduction in the size of a complete proof checker, an absolutely simplest axiomatization, or a literature-wide priority

claim. In particular, the tiny programs implement the inherited initialized-iterator theory, not $T_{F\epsilon}$. Attaching their byte counts to the new arithmetic would be incorrect. The present contribution is the new scalar arithmetic completion, its explicit interpretation proof, and small measured implementation trade-offs for the existing theory.

2 All logical rules of the new arithmetic theory

The formula grammar of $T_{F\epsilon}$ is

$$A ::= (s = t) \mid \bot \mid A \wedge A \mid A \vee A \mid A \to A \mid \forall x A \mid \exists x A.$$

Negation means $A \to \bot$ and a biconditional means the conjunction of its two implications. Contexts are finite reusable lists of formulas. Hypothesis use gives $\Gamma \vdash A$ when $A \in \Gamma$. Weakening, exchange, contraction, capture-free numerical substitution, and alpha-renaming are allowed; they may equivalently be treated as admissible structural operations of natural deduction. Contexts from multiple premises are combined, and discharged assumptions are indicated.

Symbol	Complete introduction and elimination inventory
$\bot$	From $\Gamma \vdash \bot$ infer $\Gamma \vdash C$. There is no logical introduction rule.
$\wedge$	From A, B infer $A \wedge B$; from $A \wedge B$ infer either conjunct.
$\vee$	From A infer $A \vee B$, and from B infer $A \vee B$. From $A \vee B$, $\Gamma, A \vdash C$, and $\Gamma, B \vdash C$, infer $\Gamma \vdash C$, discharging the two branch assumptions.
$\to$	From $\Gamma, A \vdash B$ infer $\Gamma \vdash A \to B$, discharging A. From $A \to B$ and A infer B.
$\forall$	From $\Gamma \vdash A(z)$ infer $\Gamma \vdash \forall x A(x)$ when z is fresh for the retained context and fixed parameters. From $\forall x A(x)$ infer $A(t)$.
$\exists$	From $A(t)$ infer $\exists x A(x)$. From $\exists x A(x)$ and $\Gamma, A(z) \vdash C$ infer $\Gamma \vdash C$, discharging $A(z)$, with z fresh for Γ, C and fixed parameters.
$=$	Reflexivity $\Gamma \vdash t = t$. From $s = t$ and $G(s)$ infer $G(t)$ for any fixed formula template, using capture-free substitution.

An instance of a numerical axiom may be used in any retained context. These are all the logical rules. There is no excluded-middle schema for arbitrary formulas and no induction rule on arbitrary quantified matrices. Actual numerical equality becomes decidable through the explicit numerical test; that does not classicalize the full logic.

3 Scalar self-seeded iteration with actual equality

There is one numerical sort, with terms

$$t ::= x \mid 0 \mid St \mid \epsilon(t, t) \mid F(n; v.h).$$

Only h binds v. Every fixed earlier-stage body $h(v, \vec{p})$ gives one first-order symbol $F_h(n, \vec{p})$; there is no function-variable sort and no uniformly total evaluator. Ordinary intuitionistic first-order logic with actual equality is used.

The numerical axioms are

$$F(0; v.h) = S(h[0/v]), \qquad F(Sn; v.h) = h[F(n; v.h)/v],$$
$$Sx = Sy \longrightarrow x = y, \qquad 0 = S0 \longrightarrow \bot,$$
$$\epsilon(x, y) = 0 \ \vee \ \epsilon(x, y) = 1, \qquad \epsilon(x, y) = 1 \longleftrightarrow x = y.$$

Here $1 = S0$ and all free variables are universally closed. The sole induction rule is contextual induction on a literal equation:

$$\frac{\Gamma \vdash E(0) \quad \Gamma, E(z) \vdash E(Sz)}{\Gamma \vdash E(t)},$$

where z is fresh for the retained context and fixed parameters. All substitutions avoid capture. Successor injectivity is an explicitly charged additional axiom. Numerical coverage is derived below rather than postulated.

Optional definitional removal of the successor constructor. The source grammar may be shortened to $x, 0, \epsilon, F$ by defining $S(a) := F(0; v.a)$. The constant-body computation already proves this identity in the displayed theory. In the shortened presentation write both computation schemas and the boundary axioms using that abbreviation. The fixed constant-body symbol belongs to the first description stage; its own base equation is tautological, while its step equation and the boundary axioms remain substantive. Reintroducing S by the definition recovers the displayed theory, and expanding S gives the converse translation. This familiar constructor-elimination device already occurs in the supplied successor-seeded drafts; the new contribution below is the scalar self-seeded completion, not this abbreviation by itself.

3.1 Elementary Boolean operations and quantifier-free induction

Put

$$D(n, a) := F(n; v.a), \qquad Z(n) := D(n, 0).$$

The computation axioms prove

$$D(0, a) = Sa, \quad D(Sn, a) = a, \quad Z(0) = 1, \quad Z(Sn) = 0.$$

If $Sn = 0$, then $D(Sn, 0) = 0 = D(0, 0) = 1$, contradicting separation. Thus every successor is nonzero. Injectivity and this consequence distinguish every fixed pair of distinct numerals.

For Boolean b, c define

$$\neg_B b = \epsilon(b, 0), \qquad b \wedge_B c = \epsilon(D(b, \epsilon(c, 0)), 0).$$

Their four truth-table cases follow from the displayed computations. Disjunction is defined by De Morgan on these Boolean values, and implication by $\neg_B b \vee_B c$. Numerical coverage is not substituted for Boolean range: the primitive range axiom supplies the two cases of every equality characteristic, and the finite truth tables preserve that range.

For each fixed quantifier-free formula A, an external recursion on syntax produces a Boolean source term χ_A such that

$$\chi_A = 1 \longleftrightarrow A, \qquad \chi_A = 0 \longleftrightarrow \neg A.$$

The implication clause is intuitionistically valid because its quantifier-free antecedent has the proved Boolean characteristic and is therefore decidable. Equation induction on $\chi_A = 1$ gives contextual quantifier-free induction. This does not add induction for arbitrary quantified formulas.

3.2 A finite-case predecessor graph

For a passive parameter a define

$$U_a(v) := D(\epsilon(Sv, a), v), \qquad V(j, a) := F(j; v.U_a(v)), \qquad W(a) := V(a, a).$$

If $a \neq 1$, then $V(0,a) = 2$. The Boolean cases show that $U_a(v)$ is v when $Sv = a$, and Sv otherwise. Contextual quantifier-free induction on j therefore proves

$$a \neq 1 \quad \longrightarrow \quad \big(V(j,a) = S^2 j \ \vee \ S(V(j,a)) = a\big). \tag{P-invariant}$$

In the first alternative of the step, a successful test gives the second alternative and a failed test advances the clock by one. In the second alternative the update fixes the value.

Take $a = S^3 m$ and $j = m$. This a differs from 1 by injectivity and successor nonzeroness. The first alternative gives $V(m,a) = S^2 m$. The second gives the same equation by successor injectivity, since $S(V(m,a)) = S^3 m$. The update now fixes the value, so exactly three further computation steps give

$$W(S^3 m) = S^2 m. \tag{W}$$

No induction with a changing passive parameter was used: the induction proved P-invariant with a fixed, and only afterwards specialized it.

The values at the three small arguments, calculated directly, are $W(0) = 2$, $W(1) = 2$, and $W(2) = 4$. Define the graph $\Phi_P(n,p)$ by the following finite quantifier-free cases:

$$\begin{aligned}\Phi_P(n,p) := {} & (n = 0 \wedge p = 0) \vee (n = 1 \wedge p = 0) \vee (n = 2 \wedge p = 1) \\ & \vee (n \neq 0 \wedge n \neq 1 \wedge n \neq 2 \wedge p = W(n)).\end{aligned}$$

Equality decidability makes these guards exhaustive; they are pairwise disjoint. Thus the graph is provably total and single-valued, without an existence oracle or a new recursion schema. Introduce its function P in a conservative definitional extension. No constructor-coverage principle has yet been used.

Every term containing finitely many P occurrences can be expanded into finitely many quantifier-free source guards with source value terms: refine argument cases and then the four cases of Φ_P. Consequently each quantifier-free formula of the extension has a provably equivalent source quantifier-free formula. Applying source quantifier-free induction proves its extension instance. This is a finite-case argument, not the invalid assertion that every conservative definition automatically preserves a restricted induction schema.

Deriving predecessor and coverage together. Apply this legitimate quantifier-free induction in the extension to the joint invariant

$$\big(n = 0 \ \vee \ n = S(Pn)\big) \quad \wedge \quad P(Sn) = n. \tag{Joint invariant}$$

At zero the first conjunct is immediate, and the second is $P1 = 0$ from the finite cases. For the successor first conjunct, the old second conjunct gives $Sn = S(P(Sn))$. For its second conjunct $P(S^2 n) = Sn$, split the old first conjunct. If $n = 0$, use $P2 = 1$. If $n = S(Pn)$, then $S^2 n = S^3(Pn)$ is distinct from $0, 1, 2$; the last predecessor case and (W) give

$$P(S^2 n) = W(S^3(Pn)) = S^2(Pn) = Sn.$$

Thus the source proves, through finite-case elimination,

$$P0 = 0, \qquad P(Sn) = n, \qquad n = 0 \ \vee \ n = S(Pn).$$

Ordinary existential introduction now gives numerical coverage $n = 0 \vee \exists m\, n = Sm$. This is an axiom deletion, not a standard-model observation: the single displayed joint induction constructs both conclusions internally, before using coverage in any subsequent order argument.

3.3 Capped counting and an order candidate

Define

$$H_a(v) := D(\epsilon(v, S^2a), v), \qquad C(n, a) := F(n; v.H_a(v)).$$

Since $S^2a \neq 0$, $H_a(0) = 1$ and $C(0, a) = 2$. The same two-case induction as above gives

$$C(n, a) = S^2n \ \vee \ C(n, a) = S^2a, \qquad C(a, a) = S^2a. \tag{C-invariant}$$

If the cap has been reached the next state is identical. In particular $C(n, 0) = 2$ by equation induction.

Coverage proves $D(c, Sv) = S(D(c, v))$. Boolean reflection and successor injectivity prove $\epsilon(Sv, S^3a) = \epsilon(v, S^2a)$; hence $H_{Sa}(Sv) = S(H_a(v))$. Moreover $C(1, Sa) = 3$ because the cap S^3a cannot equal 2. Equation induction on n, keeping a fixed, now proves the factor identity

$$C(Sn, Sa) = S(C(n, a)). \tag{C-factor}$$

Thus the Boolean term

$$L(a, n) := \epsilon(C(n, a), S^2a)$$

satisfies the familiar comparison recurrences

$$L(0, n) = 1, \qquad L(Sa, 0) = 0, \qquad L(Sa, Sn) = L(a, n). \tag{L-recurrence}$$

Write $a \leq n$ for $L(a, n) = 1$. The orbit supplies more than its two-variable comparison recurrences. Its persistence and the invariant give the unilateral law

$$a \leq Sb \quad \longleftrightarrow \quad (a \leq b \ \vee \ a = Sb). \tag{New-cap}$$

Indeed, if the cap is already reached it persists. If it is not reached, the old value is S^2b, the next value is S^3b, and this equals the cap S^2a exactly when $a = Sb$. The reverse direction uses persistence and $C(a, a) = S^2a$.

Linearity now has a valid fixed-parameter induction. Fix a and induct on n in $a \leq n \vee n \leq a$. The base uses $0 \leq a$. At the successor, $a \leq n$ persists. In the other alternative split $a = 0$ or $a = Sb$ by coverage. The zero case gives $n = 0$. In the successor case New-cap gives $n \leq b$ or $n = a$. The first gives $Sn \leq Sb = a$ by the factor identity; the second gives $a \leq Sn$ by reflexivity and persistence. The induction hypothesis has kept a fixed throughout.

For completeness, $Sn \not\leq n$ follows by equation induction from $L(1, 0) = 0$ and the factor recurrence. The implication $Sn \leq a \Rightarrow n \leq a$ follows by splitting $a = 0$ or $a = Sb$, using the factor recurrence and persistence. Antisymmetry is induction on n, with a fixed, in $(a \leq n \wedge n \leq a) \to a = n$. At a successor New-cap splits $a \leq Sn$ into $a = Sn$ or $a \leq n$; in the latter case $Sn \leq a$ implies $n \leq a$, the induction hypothesis gives $a = n$, and $Sn \not\leq n$ gives a contradiction. Transitivity is induction on c, with a, b fixed, in $(a \leq b \wedge b \leq c) \to a \leq c$. New-cap splits the new second comparison into its old one or $b = Sc$. These give the conclusion by the induction hypothesis and persistence, or by substitution. Thus $\leq$ is an internal discrete linear order with least element zero. Successor preserves and reflects it by L-recurrence.

The finite-case graph choosing n when $n \leq a$ and a otherwise defines $\min(n, a)$. C-invariant, antisymmetry, and the definition of L prove

$$C(n, a) = S^2 \min(n, a). \tag{C-min}$$

All subsequent uses of order and minimum refer to these derived graphs and laws, not standard-model interpretations.

3.4 Conditional bounded selection: exact local lemma

Let $M(c,p,q) = (c \wedge_B p) \vee_B (\neg_B c \wedge_B q)$, for Boolean c, p, q. Define the following source terms:

$$T(b,c,a,d) := F(b; v.D(M(c, \epsilon(v, S^2 a), \epsilon(v, S^2 d)), v)),$$
$$\mathrm{Sel}(b,c,a,d) := W(W(T(b,c,a,d))).$$

If $c = 1$, recurrence uniqueness gives $T(b,c,a,d) = C(b,a)$; if $c = 0$, it gives $T(b,c,a,d) = C(b,d)$. Accordingly the exact proved conditional hypotheses

$$a = S^2 a_0,\ d = S^2 d_0,\ L(a,b) = 1,\ L(d,b) = 1$$

imply that Sel returns a when $c = 1$ and d when $c = 0$. Indeed the selected capped value is $S^2 a$ or $S^2 d$, and both applications of W are in the range covered by (W), so they remove exactly two successors. This lemma uses the literal equations $L(a,b) = L(d,b) = 1$, not unproved order properties.

The stronger branch-local version is also valid: when $c = 1$ only $2 \le a \le b$ is needed, and when $c = 0$ only $2 \le d \le b$ is needed. The Boolean mux erases the other test pointwise; recurrence uniqueness then identifies T with the selected capped counter. The compiler below nevertheless makes both candidates bounded before using Sel, to keep its invariant uniform.

4 A source compiler for bounded scalar iteration

This section is the step that allows graph-defined operations to be used in updates without enlarging the primitive recursor schema.

4.1 Finite cases and encoded values

Call a total single-valued function graph *finite-case* if it has finitely many source quantifier-free guards $A_i(\vec{x})$ with source value terms $t_i(\vec{x})$, the guards are provably exhaustive, and the values agree whenever their guards overlap. Equivalently its graph is $\bigvee_i (A_i(\vec{x}) \wedge y = t_i(\vec{x}))$. Finite composition of such graphs again has this form, by taking finite common refinements of the argument cases. The same expansion eliminates the corresponding graph-defined function symbols from every fixed quantifier-free formula. Hence quantifier-free induction transfers to their conservative definitional extension. This assertion concerns ordinary function terms only: it does not freely admit a new primitive F body containing an opaque defined function.

Write $e(x) = S^2 x$. On encoded inputs $u = e(x)$ the following finite source cases decode x:

$$u = 2 :\ 0; \qquad u = 3 :\ 1; \qquad u \neq 2,3 :\ W(W(u)). \tag{Decode}$$

The last case has $u \ge 4$ by coverage, and two applications of (W) therefore remove exactly two successors. No source term is claimed to be a correct decoder at all encoded values without these cases.

For $B \ge 2$ define the source clipping term

$$K_B(v) := W(W(C(v,B))).$$

When $v \ge 2$, C-min and (W) prove

$$K_B(v) = \min(v, B), \qquad 2 \le K_B(v) \le B. \tag{Clip}$$

Only arguments of the form $e(t)$ or values already in $[2, B]$ are supplied to this term in the following compiler. There is no assertion that Clip has the same description at $v = 0, 1$.

Let $g(\vec{p})$ and $h(v,\vec{p})$ be fixed finite-case graphs. Fix a raw cap b and put $B = e(b)$. Expand the finite cases of g, encode and clip each candidate value directly as $C(t_i, b) = e(\min(t_i, b))$, and fold the cases by nested $\mathrm{Sel}(B, \chi_{A_i}, -, -)$, with default value 2. This gives a source term $a_B(\vec{p})$ in $[2, B]$ that equals $e(\min(g(\vec{p}), b))$.

For the update, first compose the finite cases of h with Decode for its state argument, refine the resulting source guards, and perform the same encoded clipping and selection. The result is a literal original-language source term $h_B(u, \vec{p})$, provably in $[2, B]$ for every numerical u, such that

$$h_B(e(v), \vec{p}) = e(\min(h(v, \vec{p}), b)). \qquad \text{(Compiled update)}$$

The universal range follows because every candidate is $C(t_i, b)$ and hence lies in $[2, B]$, including at inputs not satisfying its guard. Equality characteristics and Boolean connectives make every guard test Boolean; no lazy-evaluation assumption is involved.

4.2 Two transient sentinels

Define the source term

$$\begin{aligned} H_B(u) := \mathrm{Sel}\big(S^2B, \epsilon(u, 0), B,\\ \mathrm{Sel}\big(S^2B, \epsilon(u, SB), S^2B, \mathrm{Sel}(S^2B, \epsilon(u, S^2B), a_B, h_B(u))\big)\big). \end{aligned}$$

Every candidate of these selectors lies in $[2, S^2B]$. Since the sentinels SB, S^2B are distinct and exceed B, the theory proves

$$H_B(0) = B, \quad H_B(SB) = S^2B, \quad H_B(S^2B) = a_B, \quad 2 \le u \le B \Longrightarrow H_B(u) = h_B(u).$$

Consequently

$$F(0; u.H_B) = SB, \quad F(1; u.H_B) = S^2B, \quad F(2; u.H_B) = a_B.$$

Set

$$Y_b(n, \vec{p}) := F(e(n); u.H_B(u)).$$

Quantifier-free induction proves $2 \le Y_b(n, \vec{p}) \le B$. Its base and step equations are

$$Y_b(0, \vec{p}) = a_B, \qquad Y_b(Sn, \vec{p}) = h_B(Y_b(n, \vec{p}), \vec{p}).$$

Applying the finite Decode cases to Y_b produces a finite-case graph $R_b(n, \vec{p})$ satisfying

$$\begin{aligned} R_b(0, \vec{p}) &= \min(g(\vec{p}), b),\\ R_b(Sn, \vec{p}) &= \min(h(R_b(n, \vec{p}), \vec{p}), b), \qquad \text{(Bounded iteration)}\\ R_b(n, \vec{p}) &\le b. \end{aligned}$$

Every occurrence of F in Y_b has a literal source body. In particular, the finitely many auxiliary graph functions do not occur as unexplained new symbols inside that body. The construction is an external finite compiler from their cases to source syntax.

Scope of the lemma. This gives arbitrary initialized, capped *scalar* iteration of finite-case graph functions. It does not yet assert unbounded recursion totality. Below each unbounded use is assigned a source unary cap and supplied with an internal proof that clipping cannot occur on its intended arguments.

5 Arithmetic graphs without a circular size bound

All graph-defined functions in this section are finite-case, and every induction mentioned is quantifier-free after their finite-case elimination. They do not become additional permissible F bodies. The compiler of the preceding section is used whenever an iteration update refers to one of these graphs.

5.1 Difference and doubling

Use Bounded iteration with initializer x, update P, and cap x. Since $P(v) \le v$, clipping is never active. This defines a total finite-case graph $d(x, n)$ with

$$d(x, 0) = x, \qquad d(x, Sn) = P(d(x, n)), \qquad d(x, n) \le x.$$

Equation induction on y, with x fixed, proves $d(Sx, Sy) = d(x, y)$: its base is $P(Sx) = x$, and its step applies P to the previous equality. In particular $d(n, n) = 0$ by a further equation induction. No interpretation as subtraction is assumed at this stage.

Let

$$Q(n) := F(n; v.S^2 v), \qquad \mathrm{db}(n) := P^3(Q(n)).$$

Here Q is a source term, and db is a finite-case graph. Their equations are

$$Q(0) = 3, \quad Q(Sn) = S^2 Q(n), \qquad \mathrm{db}(0) = 0, \quad \mathrm{db}(Sn) = S^2\,\mathrm{db}(n), \quad Q(n) = S^3\,\mathrm{db}(n).$$

For the last three identities, first prove $Q(n) \ge 3$ by induction; functional predecessor coverage then permits cancelling precisely three successors, and the displayed recurrence follows. Quantifier-free induction proves $n \le \mathrm{db}(n)$ and monotonicity. For monotonicity induct on the larger argument in $a \le b \to \mathrm{db}(a) \le \mathrm{db}(b)$ and use the equality-or-predecessor alternatives in New-cap.

5.2 Addition by a saturated diagonal argument

For any cap c, let $A_c(n, a)$ be Bounded iteration with initializer a and update S. Write $s_c(v) = \min(Sv, c)$. Thus

$$A_c(0, a) = \min(a, c), \qquad A_c(Sn, a) = s_c(A_c(n, a)).$$

For $a \le c$, equation induction gives seed commutation:

$$A_c(n, s_c(a)) = s_c(A_c(n, a)). \qquad \text{(Seed commute)}$$

The two sides have the same base and the same update. The finite-case minimum graph obeys $\min(0, c) = 0$ and $\min(Sn, c) = s_c(\min(n, c))$, by the discrete linear-order laws. Iteration uniqueness therefore gives $A_c(n, 0) = \min(n, c)$.

Fix $a \le c$ and induct on n in

$$n \le c \;\longrightarrow\; A_c(n, a) = A_c(a, n).$$

The base follows from the preceding minimum identity. Whenever $Sn \le c$, the step applies s_c on the first side, and Seed commute on the second; the induction hypothesis then identifies them. This proves commutativity for bounded arguments without assuming addition. The increasing update and order-preserving s_c also prove that A_c is monotone in its counter and in its initializer within $[0, c]$. Counter monotonicity uses induction on the larger counter and New-cap; initializer monotonicity is direct induction on the common counter.

Now fix an input bound b and choose the *source* cap $c = Q(b) = S^3\,\mathrm{db}(b)$. Quantifier-free induction on n in

$$n \le b \;\longrightarrow\; A_c(n,n) = \mathrm{db}(n)$$

proves the diagonal identity. At a successor Seed commute makes $A_c(Sn,Sn) = s_c^2(A_c(n,n))$. Since $\mathrm{db}(Sn) \le \mathrm{db}(b) < c$, neither of these two successor operations is clipped. Thus the successor diagonal equals $S^2\,\mathrm{db}(n) = \mathrm{db}(Sn)$.

Monotonicity now gives the global local bound

$$n, a \le b \quad\Longrightarrow\quad A_{Q(b)}(n,a) \le A_{Q(b)}(b,b) = \mathrm{db}(b) < Q(b). \qquad \text{(Addition bound)}$$

So these runs have no clipping on bounded inputs. For two caps from input bounds b, b', induction on n restricted by $n \le b, b'$ proves that the results agree whenever $a \le b, b'$: the two bases are a, and both successor steps are genuine successor operations.

Choose b as the first largest of the two arguments, by the finitely many source order cases. This defines a finite-case graph $n{+}a$. Coherence proves its defining equations $0{+}a = a$ and $(Sn){+}a = S(n{+}a)$ across possible changes of chosen cap. The graph is commutative by bounded commutativity. It has the source unary bound $Q(b)$ on arguments at most b. Usual associativity and monotonicity follow by fixed-parameter quantifier-free inductions from these equations.

5.3 Multiplication and primitive-recursive pairing

Define the source unary function

$$E(n) := F(n; v.Q(v)).$$

It satisfies $E0 = 4$, $E(Sn) = Q(E(n)) = \mathrm{db}(E(n)) + 3$, is monotone, and $E(n) \ge n$. Fix $n, a \le b$ and apply Bounded iteration with initial value zero, update $v \mapsto v + a$, and source cap $E(Q(b))$. For the run at stage $j \le b$, use the quantifier-free invariant

$$v_j \le E(b + j).$$

Its base is immediate. At a successor,

$$v_j + a \le E(b+j) + b \le \mathrm{db}(E(b+j)) < E(S(b+j)).$$

Here $E(b+j) \ge b$ and the already constructed addition is used only through its finite-case graph. Also $b{+}j \le \mathrm{db}(b) < Q(b)$, so the stage bound is at most the chosen cap. Thus clipping is inactive throughout the intended run. The same cap-coherence induction as for addition gives a finite-case multiplication graph, with source unary bound

$$M_\times(b) = E(Q(b)).$$

Its zero and successor equations give the elementary nonnegative semiring identities by quantifier-free induction. In particular, $w^2 \ge w$ for $w \ge 1$ and $w^2 + w < (Sw)^2$.

Put

$$\kappa(x,y) = (x+y)^2 + x.$$

This is a finite-case graph, with a monotone inflationary source bound

$$K(b) = Q(M_\times(Q(b))).$$

Both coordinates of a valid code are no greater than the code. To construct its projections, use capped scalar iteration starting at zero with update

$$u \longmapsto \begin{cases} Su, & (Su)^2 \le z, \\ u, & \text{otherwise}, \end{cases}$$

cap z, and counter z. The update preserves $u \le z$ and $u^2 \le z$. The additional quantifier-free invariant

$$u_j = j \quad \vee \quad \text{the update fixes } u_j$$

shows that the state at $j = z$ is fixed: if it equals z, then $(Sz)^2 > z$ already makes it fixed. Hence the result $r(z)$ satisfies $r(z)^2 \le z < (Sr(z))^2$. Linear order and square monotonicity make such a result unique.

Set

$$\pi_0(z) = d(z, r(z)^2), \qquad \pi_1(z) = d(r(z), \pi_0(z)).$$

The difference identity $d(x + y, y) = x$ follows by equation induction on y, using $d(Sx, Sy) = d(x, y)$ and the addition equations. For $z = \kappa(x, y)$, put $w = x + y$. The inequalities $w^2 \le z \le w^2 + w < (Sw)^2$ give $r(z) = w$, whence

$$\pi_0(\kappa(x, y)) = x, \qquad \pi_1(\kappa(x, y)) = y.$$

The graph functions are total on arbitrary codes too, and $\pi_0(z), \pi_1(z) \le z$, since each difference is bounded by its first argument. No unbounded minimization has been introduced.

6 Full primitive recursion and both interpretations

6.1 A source bound for every fixed description

We externally recurse on ordinary finite primitive-recursive descriptions. The induction data for f are a finite-case graph, internal proofs of its defining equations, and a monotone inflationary *source unary term* M_f such that

$$\vec{x} \le b \quad \Longrightarrow \quad f(\vec{x}) \le M_f(b).$$

Zero, successor, and projections have these data with the bound Q. Composition takes finite common refinements of cases. If $f = h(g_1, \ldots, g_m)$, the composition $M_h \circ M_{g_1} \circ \cdots \circ M_{g_m}$ is a suitable source bound: monotonicity and inflationarity make it dominate each needed intermediate argument. Empty tuples and nullary descriptions are handled by their fixed finite source values and inflationary constant shifts of b.

Suppose the next description is

$$f(0, \vec{p}) = g(\vec{p}), \qquad f(Sn, \vec{p}) = h(n, f(n, \vec{p}), \vec{p}).$$

Let $G = M_g$, $H = M_h$, and choose the source unary function

$$D = K \circ H \circ G \circ Q, \qquad U(t) = F(t; v.D(v)), \qquad B(b) = U(SQ(b)).$$

Every factor is monotone and inflationary, and $D(v) \ge Sv$. Consequently U is monotone and $U(t) \ge St$; B is a monotone inflationary source term. These facts are quantifier-free inductions using the already proved order laws.

The scalar state update is the finite-case graph

$$J(z, \vec{p}) = \kappa\big(S\pi_0(z), h(\pi_0(z), \pi_1(z), \vec{p})\big),$$

with initializer $\kappa(0, g(\vec{p}))$. Apply the bounded source compiler to this initializer and update with cap $B(b)$. Assume $n, \vec{p} \le b$. For stages $j \le b$ prove

$$z_j \le U(Sb + j). \qquad \text{(PR-stage)}$$

The seed satisfies

$$\kappa(0, g(\vec{p})) \le K(G(b)) \le D(b) \le D(U(b)) = U(Sb).$$

For the successor, write $s = U(Sb + j)$. Then $s \ge b$, and both state projections are at most $z_j \le s$. The next counter is at most $Ss \le Q(s)$, and the new value is at most $H(G(Q(s)))$, by monotonicity and inflationarity of the bounds. The pairing bound therefore gives

$$J(z_j, \vec{p}) \le D(s) = U(S(Sb + j)).$$

Since $j \le b$, the stage argument $Sb + j$ is at most $S\,\mathrm{db}(b)$ and hence at most $SQ(b)$. Thus every stage bound is at most $B(b)$, proving that no clipping occurs. The displayed invariant is quantifier free in the finite-case definitional extension; it translates to permitted source induction.

The projection identities prove by quantifier-free induction guarded by $j \le b$ that the counter component of z_j is j, and that the value component satisfies its required base and, when $Sj \le b$, step equations. Two valid caps give the same run up to their common allowed counter by induction: the seeds agree and their unclipped updates are identical. Choose b as the first largest input among $n, \vec{p}$, by finitely many source order cases, and define $f(n, \vec{p}) = \pi_1(z_n)$. This is again a finite-case graph. At any larger input bound b, cap coherence identifies this value with the run at cap $B(b)$, or monotonicity of B bounds it from the selected smaller input bound. Thus its value is bounded by $B(b)$ for every valid input bound, and take $M_f = B$. The cap-coherence argument proves its defining equations even when a successor input changes the selected largest coordinate.

The source compiler has expanded every use of g, h, κ, π_i inside the scalar update into original-language terms, encoded cases, and bounded selectors. Neither a total evaluator for description codes nor a new recursor schema over graph-defined operations has been assumed. The construction is a finite external recursion on each fixed description.

6.2 Whole-domain bi-interpretation

Interpret the source theory in iPRA on its full numerical domain. Keep $0, S$ and actual equality; take ϵ to be the standard primitive-recursive equality characteristic. For each fixed earlier body h, interpret F_h by primitive recursion with seed $Sh(0)$ and scalar update h. All source axioms and its equation-induction rule then hold in iPRA.

Conversely, interpret each fixed PR function description by the finite-case graph constructed above. Its totality, single-valuedness, defining equations, and quantifier-free induction instances have all been proved in the source. The original successor, injectivity, separation, and actual equality are retained. Thus this is an interpretation of full first-order iPRA, with its ordinary intuitionistic logic and the entire numerical domain.

For the iPRA-side composite, external induction on a fixed PR description reduces identification with the original function to uniqueness from its base and step equations. Internal quantifier-free induction proves this uniqueness at arbitrary counters and parameters. For the source-side composite, $0, S$ are unchanged. Both equality characteristics have the two-value range and reflect actual equality, so Boolean cases identify them. For a fixed F_h, external induction on its finite stage identifies the earlier body; both the old and composite functions then satisfy the same self-seeded base and scalar step equations. Equation induction identifies them everywhere. If a composite uses graph-defined functions, the finite-case elimination lemma supplies the required source induction instance.

The relation $x = y$ therefore defines the isomorphism of each domain composite with the original structure. External induction on formula syntax extends preservation of functions and equality to all formulas; quantifiers range over the same whole domain and genuine falsity is preserved. This gives whole-domain bi-interpretability and equiconsistency. It is not an internal assertion of either theory's own consistency.

Research status and complexity. This is a written proof of a new scalar self-seeded basis, with the separately recorded independent mathematical audit. It removes both the independent initializer and the old counter argument from the primitive recursor. It pays explicitly for actual numerical equality, successor injectivity, and ordinary intuitionistic logic. No complete checker for this new theory has been measured, and no existing tiny initialized-iterator checker is assigned to it. It must not be described as a demonstrated improvement below the current complete-checker frontier or as an absolutely simplest theory.

7 The separate complete-checker result

The measured programs implement the inherited Choice-DI presentation with initialized scalar iteration

$$I(0, a; v.h) = a, \qquad I(Sn, a; v.h) = h(I(n, a; v.h)).$$

Its native formulas are equations, paired binders $\mathsf{D}_{x,y}A := \exists x(A(x, 0) \to \forall y A(x, y))$, and auxiliary closures $\mathsf{Q}_y A := A(0) \to \forall y A(y)$. Its complete rule inventory, logical reconstruction, scalar predecessor, equality reflection, and whole-domain bi-interpretation are reproduced with attribution in the appendices. None of that inherited arithmetic is relabeled as a new result.

The implementation combines the lexical proof scopes of [4], the arity encoding of [5], and the initializer dispatch of [3]. The prior dispatcher uses a proved nonempty representation $C = I(2, 1; v.0) = 0$ to recover computation instances with an omitted literal-zero initializer or predecessor. That normalization is inherited in full.

7.1 A complete readable-source profile

The following 562-byte explicit-list program is an entire executable. Its 554-byte sibling replaces the list test with the inherited zero-product predicate. No theorem database, test producer, evaluator, or translator is imported by either program.

```
import json

def c(a, g, k=0):
    i = lambda a, t=None, d=0: [i(b, t, d + 5 - len(a)) for b in a] if isinstance(a, list) else a if k
    <= a < d else d % 2 and i(t, d=3)
    if i(a) in g:
        return
    m, s, u, t = q()
    if m:
        c(m, g, k)
        a = m in ([s, u], not s and [t] * 3) and i(t, u) == a and i(t, s) or a
    h = m * 3 if m else [[s and [s], u, t], i(t, [s, u, t], 3) if s else u] if u else (a := i(*a, t, 2)
    )
    k -= 1
    c(i(a, [k]), g + [i(h, (m or len(a) % 2 == m) and k)], k)
q = json.load(open(0)).pop
c(q(), [])
print('ACCEPT')
```

The shared execution environment is Python and its standard JSON library, as in the inspected comparisons. Every other byte is counted, including loading input, opening binders, recognizing numerical and logical instructions, and printing the verdict.

7.2 Local freshness and context isolation

At a call $c(a, g, k)$, the local floor k is the negative of the number of retained assumptions on that branch. Opening validates the goal at this floor. A major subproof receives the same g, k; its private changes to k cannot affect its caller. A continuation decreases its own floor by one, opens the continuation and the supplied assumption, and appends that assumption to a fresh list.

This is the existing lexical-scope invariant, now composed with the existing certified-zero dispatcher. The source and AST of the composition do not occur verbatim in the frozen corpus, but its ingredients do. Every finite canonical native derivation is serialized after branchwise alpha-renaming to these local negative names. The old-floor check prevents a meaningful retained goal or hypothesis from mentioning a parameter before it is allocated. Disjoint subproofs may reuse names after a private subproof returns.

Soundness is induction on a successful checking trace, using the inherited semantic opening and numerical zero-instance lemmas. A major branch uses its proved major and either transport, closure specialization, or paired elimination. An introduction uses the unchanged literal-equation restriction on progress. Computation uses a derivable zero instance of the generated hypothesis; it is not assumed true at every possible new parameter. Substitution of zero for its fresh names and assumption cut justify the continuation. Completeness follows from inherited computation normalization and branchwise renaming of the canonical proof tree.

The loop variants replace tail recursion by a loop and retain contexts as immutable tuples. Rebinding g in a subproof does not change the caller's tuple. Replacing this by in-place list mutation is unsound: the included control certificate proves $0 = S0$ in that mutation by leaking a subproof's private assumption. The correct programs reject it.

7.3 Input and resource qualifications

A certificate is a JSON array whose last element is its closed goal; earlier four-field instructions are consumed backwards. An unused prefix is inert. Acceptance requires both exit status zero and the exact output line `ACCEPT`; a traceback or a merely appearing word is not acceptance. The programs contain no trusted `assert` statement, so optimized Python does not disable their checks.

The explicit-list and zero-product-list profiles use structurally recursive traversal on finite JSON syntax. The generic traversal in the two loop profiles has an inherited qualification: malformed nonempty strings can recurse until the real Python stack limit rejects them. An ideal interpreter without that limit could diverge. The AST comparison includes this qualification; it is not a claim of an equally small total abstract parser. Ordinary finite machine resources can reject sufficiently large valid proofs in any profile.

8 Measurement, tests, and evidence

The comparison uses CPython 3.12.14 and zlib 1.3.2. Genuine module, class, and function docstrings are removed; the AST is normalized with `ast.unparse`, followed by one newline. The metrics count normalized UTF-8 bytes, significant Python tokens, and all AST nodes. Each exact source is also stored with `gzip.compress(source, compresslevel=9, mtime=0)`. The gzip wrapper is included. These are FYI figures, not a cost function used to select the mathematical theory.

- All 443 frozen Python source hashes are rechecked and remeasured offline. The corpus metadata records the source archive and original member paths; supporting utilities are excluded from checker minima.

- An inherited named proof producer, with the lexical naming wrapper, regenerates the 31 canonical positive and negative cases. It imports no candidate checker. Four complete profiles, both ordinary and optimized Python, plus malformed inputs and a concrete false-proof attack give 332 checked CLI expectations: 120 accepts and 212 rejects, all as expected.
- Independent source-arithmetic diagnostics give 5,509 expectations for Boolean operations, the predecessor invariant, capped counting, comparison, bounded selection, clipping, and two-sentinel iteration. The separate reviewing agent checked another 1,248 literal macro cases. These are finite standard-model checks, not proofs for nonstandard inputs.
- The separate review records the source hash of the 28,933-byte new arithmetic proof, its coverage-axiom deletion, the finite-case induction argument, order and cap coherence, bounded source-body compilation, PR majorants, and both domain composites. It is a same-session assistant review, not external peer review or a proof-assistant certificate.

Run `python3reproduce.py` from the extracted directory to repeat the packaged measurements and finite tests. Run `python3build.py` with PDFLaTeX installed to regenerate the standalone TeX source and PDF. Neither command requires a network connection. Runtime changes may change normalized or compressed lengths, so use the recorded environment for exact size reproduction. The manifest describes the delivered files; perform experimental edits in a working copy.

9 A discarded freshness shortcut and its exact failure

An investigated shortcut replaced the integer eigenparameter floor by fresh opaque Python objects, and treated other noninteger leaves as rigid numerical parameters. That prototype is unsound. A floating-point leaf `3.0` was rigid while integer `3` denoted a bound coordinate, yet Python compared the two arrays as equal. The supplied six-command certificate exploits this difference to prove $0 = S0$. The defective program accepts it, while every selected complete checker rejects it.

The defect is a failure of compatibility between raw equality and opening. Objects considered equal by proof matching cannot have different binding behavior. Restricting opaque recognition to exact object instances blocks that particular attack, but the unchanged flat JSON stream still cannot name a newly allocated opaque object in later commands. Even universal reflexivity loses its required canonical proof. A scoped continuation-opening protocol or explicit environment is needed, and that general route already appears among prior experimental kernels. The files under `tests/failed_opaque/` are deliberately labeled failed experiments and are not candidate checkers.

10 Further research questions

1. **Can successor injectivity be removed from $T_{F\epsilon}$?** The current predecessor-tail proof uses cancellation of a successor, so deriving injectivity from that proof would be circular. Removing this remaining price of scalar self-seeding would improve the comparison with the existing B_ϵ basis.
2. **Can the new arithmetic lower complete-checker complexity?** A native logical presentation must account explicitly for equality reflection, Boolean range, injectivity, genuine falsity, and the source grammar. A complete implementation is required before assigning it a size; counting only the shorter recursor is insufficient.
3. **Can finite-case compilation be made economical?** The proof permits finite refinements and repeated bounded selection. Its size may grow rapidly with the PR description. A compiler with

explicit size bounds, shared subdescriptions, and a verified cap invariant would clarify the practical cost of this representation.

4. **Which lemma should be formalized first?** The joint predecessor/coverage induction and the bounded two-sentinel compiler are compact targets. Formalizing the finite-case elimination lemma would secure the passage from literal-equation induction to the graph-defined intermediate arithmetic without extending recursion schemas illicitly.

5. **Can one straightforward parser improve all coordinates?** The measured family is a Pareto trade-off. The remaining implementation target is a program below 554 bytes, 262 tokens, and 291 AST nodes at once, with a structurally terminating explicit parser and the same complete trusted boundary.

A Attribution and the complete inherited checker theory

The following appendices are included to make the mathematical theory implemented by the measured programs reviewable. They come from the current certified-zero archive, where they retain their earlier attribution through the arity, generated-assumptions, and scalar developments. Statements such as "new" inside these historical appendices refer to their original reports. They are not claims of novelty in this report. Only the checker listing's file path is adjusted to point at the frozen baseline. The original MIT No Attribution license and exact source provenance accompany the package.

B Using the initializer to select computation

The initialized scalar iterator has the usual equations

$$I(0, a; v.h) = a, \qquad I(Sn, a; v.h) = h[I(n, a; v.h)/v]. \tag{I0, IS}$$

The following restriction removes a computation opcode from the certificate interface without changing these operations or the native consequence relation. It concerns the *syntax* of an initializer, not whether that initializer denotes a positive natural number.

A fixed nonempty representation of zero. Let

$$C := I(S(S0), S0; v.0). \tag{Auxiliary zero}$$

One successor computation, at predecessor $S0$, proves

$$C = 0. \tag{Zero certificate}$$

The predecessor and initializer in this computation are both nonempty canonical terms. The update is the constant zero, so the right side of the computation is exactly zero. This construction does not use a previous computation with initializer zero, a zero-predecessor repair, an induction, or a presumed identity for a new numerical constructor.

Restricted computation system. In canonical numerical serialization, use scalar 0 for zero and nonempty arrays for successor and iteration. Consider the same native calculus with its direct computation leaves restricted as follows:

- I0 may be used directly only when its initializer is not literal zero.
- IS may be used directly only when both its initializer and its supplied predecessor are not literal zero.

The predecessor restriction is already present in the inherited predecessor-specified interface. The new restriction is on the initializer. A parameter or a nonempty term is permitted even when its value happens to be zero. In particular, C is permitted. Keep the ordinary native equality-transport rule and all other logical rules unchanged.

Admissibility theorem. Every unrestricted I0 and IS instance has a finite derivation in this restricted system. Consequently the restricted and unrestricted calculi prove exactly the same native sequents, with exactly the same numerical and formula languages.

Proof. Instances satisfying the stated syntactic conditions are direct. For a base computation with initializer zero, take the fresh-variable equation motive

$$E(w) := \big(I(0, w; v.h) = w\big).$$

The restricted base equation proves $E(C)$. Equality transport along the independently proved $C = 0$ gives $E(0)$, which is precisely the required unrestricted base instance.

For a successor computation with permitted predecessor n but zero initializer, take

$$E(w) := \left(I(Sn, w; v.h) = h[I(n, w; v.h)/v]\right).$$

The restricted successor equation proves $E(C)$, and the same transport gives $E(0)$. The displayed variable w is fresh for n, h, and all retained assumptions. Only the indicated initializer occurrences are substituted; unrelated zero occurrences or passive arguments in h remain fixed.

For a zero predecessor but a permitted initializer a, use instead

$$E(w) := \left(I(Sw, a; v.h) = h[I(w, a; v.h)/v]\right).$$

Again $E(C)$ is a permitted successor computation and transport gives the desired instance $E(0)$.

If both predecessor and initializer are zero, a single shared motive repairs them simultaneously:

$$E(w) := \left(I(Sw, w; v.h) = h[I(w, w; v.h)/v]\right).$$

Both supplied values in $E(C)$ are nonempty, and transport along $C = 0$ gives exactly $E(0)$. Bound state variables are renamed before these substitutions, so no capture occurs.

Each repaired computation leaf therefore uses one permitted computation for $C = 0$, one permitted computation for $E(C)$, and one transport inference. Replacing computation leaves in a native derivation proves the inclusion from the unrestricted calculus to the restricted one; the reverse inclusion is immediate. Contexts and final sequents are unchanged throughout. □

This is a three-node repair when computation leaves and transport are counted as one node each and the proof of $C = 0$ is copied. If a tree derivation has N nodes and r excluded computation leaves, the stated replacement has exactly $N + 2r$ nodes. If a DAG proof retains one shared node proving the closed lemma $C = 0$, the repaired part instead has $2r + 1$ nodes, giving $N + r + 1$ distinct nodes when $r > 0$. A certificate format with persistent proved assumptions can implement corresponding sharing by supplying this closed computation once before the affected continuation. These are counts for the displayed construction, not lower bounds on shortest certificates or claims about a particular producer's serialization.

Why the opcode can disappear. Write a four-field packet as $[m, s, u, t]$. The established elimination branch uses a truthy major m. In a computation packet, let s be the predecessor flag/value, u the initializer, and t the update. A truthy u selects the computation branch. The checker constructs the source counter as s when it is the zero flag and as Ss otherwise, and generates I0 or IS. A paired-introduction packet puts zero in the unused u field. Thus the dispatch can be written

```
if m:
    ... elimination ...
elif u:
    ... computation ...
else:
    ... paired introduction ...
```

The direct computation branch can no longer receive a literal-zero initializer. The admissibility theorem supplies exactly that missing case. No test that u denotes a nonzero number is performed or needed.

The fresh-name obligation on noncanonical packets. The dispatch change admits computation packets with any false-valued m, including the atomic-progress marker $m = 0$. In the common continuation, that marker can cause the generated hypothesis to be opened at the new rigid parameter b, rather than at literal zero. It would be incorrect to assert that the resulting $H(b)$ is uniformly true. Use the existing zero-instance invariant instead:

$$\vdash H(\vec{0}), \qquad \vec{b} \cap \mathrm{FV}(\Gamma, A) = \varnothing.$$

Here $\vec{b}$ includes the designated new integer name b and every additional negative fractional name in the newly admitted unit interval that occurs in the finite raw certificate. Such names must not be silently identified with the designated integer. Given a finite derivation of $\Gamma, H(\vec{b}) \vdash A$, rename its later eigenparameters if necessary, substitute zero for every member of $\vec{b}$, and cut the resulting assumption $H(\vec{0})$ against its computation derivation. This gives $\Gamma \vdash A$. The old-floor validation of the retained goal and context is what establishes the displayed freshness condition. Consequently the new raw packet possibilities are sound under the same zero-completion and opening lemmas as the predecessor-specified kernel. They do not justify silently strengthening a known zero instance into a uniform computation theorem.

Interpretations and the scope of the improvement. The theorem preserves every native sequent through the identity map on object expressions. It therefore transfers the inherited whole-domain bi-interpretation and equiconsistency with iPRA without a new domain, new equality relation, or new primitive numerical operation. The contribution is a removable syntactic restriction and a data-driven certificate dispatch. It is an implementation simplification, not a new arithmetic foundation or a proof of absolute minimality.

Relative to the inspected pinned draft collection, the new step is using the initializer itself to discriminate computation packets and proving that the resulting missing computation instances are admissible. The broader technique of replacing an unavailable zero representation by a proved nonempty representative is already used in the supplied drafts, including their predecessor and witness repairs. The present argument adapts that attributed technique to eliminate the computation opcode. No priority claim outside the inspected corpus is made.

C The complete checker and its interpretation

The recommended program below is the entire trusted Python source. No producer, evaluator, theorem database, measurement script or service is imported. It uses the inherited arity representation [5] with the new initializer dispatch and shared assumption assignment.

```
import json
k = 0
i = lambda a, t=None, d=0: [i(b, t, d + 5 - len(a)) for b in a] if isinstance(a, list) else a if k <= a
     < d else d % 2 and i(t, d=3)

def c(a, g):
    global k
    if i(a) in g:
        return
    m, s, u, t = q()
    if m:
        c(m, g)
        a = m in ([s, u], not s and [t] * 3) and i(t, u) == a and i(t, s) or a
    h = m * 3 if m else [[s and [s], u, t], i(t, [s, u, t], 3) if s else u] if u else (a := i(*a, t, 2)
    )
```

```
    k -= 1
    c(i(a, [k]), g + [i(h, (m or len(a) % 2 == m) and k)])
q = json.load(open(0)).pop
c(q(), [])
print('ACCEPT')
```

C.1 Input, result and execution boundary

Input is one JSON array. Its final element is the requested closed formula; earlier four-field packets are consumed backwards. An unused prefix is inert. A successful process exits zero and prints exactly `ACCEPT`. An input or checking exception rejects with nonzero exit status; the traceback is diagnostic. Merely finding the word `ACCEPT` in arbitrary output is not the verdict convention. There are no trusted Python `assert` statements, and the same inference checks run under `-O`.

C.2 Sorted array interpretation

Canonical native formulas use

$$\mathsf{D}_{x,y}B \longmapsto [B], \qquad s = t \longmapsto [s,t], \qquad \mathsf{Q}_y B \longmapsto [B,B,B].$$

Only the last field of a general ternary formula is its matrix; the first two are annotations. Numerical terms use scalar zero, unary successor and ternary scalar iteration. Both interpretation and canonical encoding are sorted: the same array may have different meanings at a numerical and a formula position.

Arity	Numerical meaning	Formula meaning
0	0	$0 = 0$
1	S of its child	D of its matrix
2	0; annotations ignored	Equality of the two terms
3	$I(n, a; v.h)$	Q of the last field
≥ 4	0; children ignored	$0 = 0$; children ignored

The scalar numerical coordinates $0, 1, 2$ all denote zero at the numerical root depth three. Under p protected numerical slots, coordinates $3 + 2j, 4 + 2j$ name slot j, counted from the innermost slot, for $0 \leq j < p$. Negative admitted coordinates name rigid numerical parameters. Formula root depth is $2p$, and numerical root depth is $3 + 2p$. Successfully admitted scalar padding at formula positions denotes the fixed truth $0 = 0$.

In a unary numerical successor, its child is decoded under two dummy zero slots. In a ternary iterator, counter and initializer are decoded under one dummy zero slot, while the update is decoded under its genuine state slot. In a paired formula the two slots are y, x; in a closure the one slot is y. Thus the single child-depth update

$$d \longmapsto d + 5 - \text{arity}$$

gives the required increments $4, 3, 2$ at arities $1, 2, 3$. Canonical encoding shifts variables past the indicated dummy slots. Rigid negative names do not shift. The named producer implements these conventions independently of the kernel.

At larger arities the entire expression already denotes a constant. Traversing an ignored annotation can reject it; a successful traversal does not change that constant interpretation. In particular, zero or negative depth increments inside such annotations do not authorize a claim that all raw trees are idempotently revalidated. Only the semantic opening statement at meaningful sorted roots is needed.

C.3 Opening and freshness

At global floor k, the opener preserves a scalar in the interval $k \le a < d$. At a numerical position an escaping scalar is the distinguished substitution hole, and its image is checked at numerical root depth three before insertion. A missing image at a used numerical hole rejects. At a formula position scalar padding instead erases to truth. Induction on a successfully traversed finite tree, using the preceding table, proves capture-free semantic opening at every meaningful position.

An inserted numerical image has no unbound positive coordinate at its own root. Its free parameters are rigid negative names. Hence copying it beneath extra protected slots does not capture its free parameters; its internal binders retain their indices. Raw equality of successfully checked expressions consequently implies equality of decoded meanings. This statement does not identify different numerical programs merely because they happen to have the same standard value.

Each non-hypothesis continuation decreases k. The integer new floor is the designated fresh parameter. Previously checked relevant goals and retained hypotheses mention no newly admitted name: their earlier no-image checks would have rejected such an occurrence. If noninteger JSON numbers occur, any additional names in the newly admitted interval are separate fresh parameters, not aliases of its integer endpoint. Only finitely many occur in a finite certificate. The known-zero-instance argument substitutes zero for all of them when necessary.

C.4 The three packet cases

Write a packet as $[m, s, u, t]$. A truthy major m takes priority. The checker proves it in the old context before proceeding. If it is the indicated equality $[s, u]$, and the current goal is the template opened at u, the goal changes to its opening at s. For a canonical closure $[t, t, t]$, the source field must denote the zero instance; the same operation implements QE. A mismatch leaves the goal unchanged. Either way a proved major is required, so a failed major does not become a harmless no-op.

The exported local expression is $m * 3$. A unary paired major exports its three-copy closure, with the witness opened at the fresh name. Every other nonempty array major exports a high-arity truth. Relevant scalar majors cannot create arbitrary unproved consequences: their only admitted formula interpretation is truth. The joint profile's additional iterable aliases are covered by its inherited decoder.

For a false-valued m and truthy u, the initializer selects computation. The packet constructs

$$\begin{cases} [\,[s, u, t], u\,], & s \text{ false-valued}, \\ [\,[[s], u, t],\ i(t, [s, u, t], 3)\,], & s \text{ truthy}. \end{cases}$$

The first is a zero computation; the second constructs an actual successor counter. When an update uses its state, filling it checks the entire recursive source, including its update, at numerical root depth three. This prevents an escaping update coordinate from being mistaken for an extra state variable. If the update does not use its state, the final generated equation still validates its meaningful contents. The inherited zero-instance proof applies after setting newly admitted names to zero.

When both m and u are false-valued, the packet is paired introduction. The unary goal is opened at the supplied witness t, retaining the index slot. Its successor instance becomes the next goal. Marker $m = 0$ retains the arbitrary-index instance only for a literal equation; the case marker $m = []$ retains the zero instance. Compound formulas get the zero instance. The literal-equation restriction therefore remains active. Exact unary unpacking rejects a binary equation as a paired introduction; the undersupplied empty-goal case has the harmless truth interpretation.

The last assignment shares construction of the local hypothesis across these cases. Python evaluates only the selected branch of the conditional expression. In the introduction branch the named assignment

updates the goal and hypothesis to the same opened matrix, just as the separate assignments did before hoisting. The equality-transport Boolean expression in the byte and preferred profiles likewise changes no canonical proof: object and closure matrices are nonempty arrays. If a noncanonical transported source is false-valued, the expression keeps the original goal, imposing the stronger unchanged obligation.

C.5 Why a generated hypothesis need not be universally true

For fresh names $\vec{b}$ absent from Γ, A, the following rule is admissible:

$$\vdash H(\vec{0}), \quad \Gamma, H(\vec{b}) \vdash A \quad \Longrightarrow \quad \Gamma \vdash A.$$

Substitute zero for the fresh names in the second finite derivation, renaming later eigenparameters first, and cut in the first proof. No consistency or truth oracle is used.

This matters for the new dispatch, since arithmetic can now be selected with the otherwise-progress marker $m = 0$. For example the packet data

$$m = 0,\ s = 0,\ u = 3,\ t = 0$$

can generate, at the fresh parameter b, the decoded assumption

$$H(b) : I(0, 0; v.0) = b.$$

Its zero instance is a computation theorem; its instance at one is false in the standard model. The retained goal is validated before b exists. The tests accept an independent valid continuation after this supply and reject an attempted closed false consequence. The soundness argument is the admissible fresh-name rule, not a mistaken claim that every generated open hypothesis is uniformly true.

C.6 Soundness, completeness and profiles

Theorem C.1 (Checker correspondence)**.** *Every successful finite run of the explicit-list checker has a decoded conclusion derivable in the native iPRA presentation. Every finite canonical native derivation has a finite certificate after the displayed computation normalization, under the usual unbounded-resource mathematical interpretation of the checker.*

Proof. For soundness, induct on the finite successful checking trace. Hypothesis closure uses semantic opening. The major branch uses the inductively proved major, transport or QE when their raw match holds, and DE or truth weakening for the export. Introduction uses the exact unary shape, the fresh-name invariant and the correct case/progress assumption. Computation uses its derivable zero instance and the fresh discharge lemma. The final continuation opening realizes the displayed successor or witness substitution. Hoisting the hypothesis construction preserves the evaluated branch and its state changes.

For completeness, first replace each excluded computation leaf by the three-node zero-certificate repair. Use canonical arity encodings, fresh negative labels, and the inherited backward packet serialization of Hyp, Tr, QE, DE and the two introductions. Every remaining computation has a truthy initializer and, in its successor case, a truthy predecessor. It therefore reaches the selected branch and generates its exact computation equation. Hyp closes that continuation. All retained contexts and conclusions are preserved by this finite translation. □

The byte profile's predicate `a*0 == []` distinguishes lists among successfully tested JSON values. Strings and numbers produce different zero products; mappings and null raise errors. It is the inherited

compact spelling of a list test, not proof-supplied execution. The preferred explicit list predicate makes this intent clearer.

The joint profile tries iterable traversal and falls back to scalar handling on an exception. It accepts some additional harmless iterable aliases. Nonempty strings may recurse indefinitely in an ideal unbounded-stack execution; actual CPython rejects through its recursion limit and subsequent scalar type failure. Its simultaneous record must therefore be read as a fixed-runtime comparison and canonical-proof completeness statement, not an ideal all-JSON termination theorem. The explicit-list and zero-product profiles avoid that malformed-string qualification. All three remain subject to ordinary physical resource limits on large genuine proofs, as do the supplied baseline programs.

D Complementary introductions with one local assumption

The old paired introduction retains both the zero instance and an induction instance when its matrix is an equation. Both together are unnecessary. A certificate may choose either one, provided the induction choice is restricted to a literal equation. This changes neither the formula language nor the meaning of a formula. The omitted combination is recovered by a short nested introduction–elimination construction.

D.1 Complete native syntax and rule inventory

The scalar numerical language is

$$t ::= x \mid 0 \mid St \mid I(n, a; u.h).$$

Only h binds the state variable u; n and a do not. The update may contain fixed numerical parameters. The computation axioms in every retained context are

$$I(0, a; u.h) = a, \qquad I(Sn, a; u.h) = h[I(n, a; u.h)/u]. \tag{I0, IS}$$

The argument also applies unchanged to the full fixed-description PR language with its projection, composition, and recursion equations. No numerical variable ranges over function descriptions, and no uniform evaluator is postulated.

Native formulas and reusable context clauses are

$$A ::= (s = t) \mid \mathsf{D}_{x,y}A, \qquad \langle A; y.B(y)\rangle, \quad y \notin \mathrm{FV}(A).$$

Bound variables are renamed before capture-free substitution. The two variables x, y in a paired formula are distinct. An ordinary assumption F is the constant clause $\langle H; v.F\rangle$, where v is absent from F and H is the fixed computation theorem $I(0, 0; u.0) = 0$. Thus ordinary hypothesis use is a clause Use preceded by this computation proof.

There are computation axioms and exactly the following inference patterns:

$$\frac{\Gamma \vdash A}{\Gamma \vdash B(t)}\ \mathsf{Use} \quad (\langle A; y.B(y)\rangle \in \Gamma),$$

$$\frac{\Gamma \vdash \mathsf{D}_{x,y}B(x, y) \qquad \Gamma, \langle B(a, 0); y.B(a, y)\rangle \vdash F}{\Gamma \vdash F}\ \mathsf{DE}, \qquad \frac{\Gamma \vdash s = t \qquad \Gamma \vdash F(s)}{\Gamma \vdash F(t)}\ \mathsf{Tr},$$

$$\frac{\Gamma, B(t, \eta_\varepsilon(z)) \vdash B(t, Sz)}{\Gamma \vdash \mathsf{D}_{x,y}B(x, y)}\ \mathsf{DI}_\varepsilon, \qquad \eta_0(z) = 0, \quad \eta_1(z) = z. \tag{Choice DI}$$

Here ε is a certificate choice in $\{0, 1\}$, and $\varepsilon = 1$ is permitted only when B has a literal equation root. An equation beneath a binder is not a literal equation. Call $\varepsilon = 0$ the *case choice* and $\varepsilon = 1$ the *progress*

choice. Both retain exactly one ordinary formula assumption. Compound matrices have only the case choice.

In DI the eigenparameter z is fresh for Γ, the witness t, and every fixed parameter of B. In DE the eigenparameter a is fresh for Γ, the conclusion F, and every fixed parameter of B. In transport, $F(v)$ is any fixed native formula template. Displayed local assumptions and clauses are discharged. There are no primitive reflexivity, coverage, separation, ordinary logical, or additional induction rules.

We call this calculus $\mathcal{C}$. The original comparison calculus $\mathcal{N}$ has identical syntax, computation, Use, DE, and transport, but its introduction is

$$\frac{\Gamma, B(t,0), [B(t,z)]_{\mathrm{at}} \vdash B(t,Sz)}{\Gamma \vdash \mathsf{D}_{x,y}B(x,y)}. \tag{Old DI}$$

The bracket supplies $B(t,z)$ exactly at a literal equation.

D.2 Exact equivalence and the unused-witness construction

Lemma D.1 (Structural admissibility)**.** *Weakening, exchange, contraction, renaming, ordinary-assumption cut, and capture-free numerical substitution are admissible in $\mathcal{C}$.*

Proof. Induct on derivations. Rename local eigenparameters before inserting proofs or terms. Numerical substitution preserves a formula's root constructor, so an allowed progress choice remains allowed. The selected choice does not change under substitution or cut. For ordinary-assumption cut, replace each use of its constant clause by the supplied proof. Computation schemas are closed under capture-free numerical substitution. These facts verify every case. □

Theorem D.2 (Identity-language equivalence)**.** *For every retained clause context Γ and native formula F,*

$$\Gamma \vdash_{\mathcal{C}} F \quad \Longleftrightarrow \quad \Gamma \vdash_{\mathcal{N}} F.$$

The transformations are effective on finite derivations and leave all numerical and formula syntax unchanged.

Proof. Each new introduction is an instance of the old one after weakening by the omitted local assumption. For the case choice at an equation, add the induction instance. For the progress choice, add the zero instance. Compound case introduction is already the old rule. Thus every new derivation translates to an old one.

For the reverse direction, transform an old derivation recursively. Only its atomic DI case needs a construction. Suppose its matrix is the equation $E(x,y)$ and its transformed premise is

$$\pi: \quad \Gamma, E(t,0), E(t,z) \vdash E(t,Sz).$$

The outer new inference will be case DI with witness t. It is therefore enough to prove $E(t,Sz)$ under $\Gamma, E(t,0)$.

Rename the eigenvariable in π to a fresh v. Choose a new variable u absent from $E(t,y)$ and all retained data. Under $\Gamma, E(t,0)$, progress DI with witness zero introduces

$$\mathsf{D}_{u,y}E(t,y)$$

from exactly the renamed premise $\pi[v/z]$. The bound coordinate u is deliberately unused.

Apply DE to this paired formula. Because u is unused, the opened clause is exactly

$$\langle E(t,0); y.E(t,y)\rangle;$$

it contains no new existential witness. The retained ordinary assumption $E(t,0)$ supplies its antecedent. Use at Sz gives $E(t,Sz)$, and DE closes. Finally the outer case DI introduces the original $\mathsf{D}_{x,y}E(x,y)$ with witness t. The original premise proof π has been inserted once.

An old compound introduction already is the new case choice. All other rule cases are unchanged. This completes the induction on the original derivation. □

The unused inner coordinate is essential. Introducing $\mathsf{D}_{x,y}E(x,y)$ internally and then eliminating it would give an arbitrary fresh witness, not necessarily the original t. The construction instead keeps t as a fixed parameter of $E(t,y)$ and binds only the unused coordinate u.

Proposition D.3 (Linear bound on inference nodes)**.** *Count each computation axiom, Use, DI, DE, and transport as one inference node, expanding ordinary-assumption use as a computation of H followed by Use. If an old derivation has N nodes and r atomic DI nodes, the displayed transformation has exactly*

$$N + 5r \leq 6N$$

nodes.

Proof. Each atomic old DI node is replaced by six nodes: outer case DI, inner progress DI, DE, Use of the opened clause, Use of the retained base assumption, and the computation of H needed for that last ordinary-assumption use. Its old premise subtree occurs once. All other inference nodes are copied once. □

This counts inference-tree nodes. It is not a bound on serialized JSON bytes, formula sizes, checker runtime, or the number of consumed packet cells when hypothesis closure is implicit. In the formula-context Q/D presentation, the corresponding replacement has outer case DI, inner progress DI, DE, QE, and two Hyp nodes, giving the same six-node count.

D.3 Induction and numerical cases separate cleanly

Lemma D.4 (Contextual equation induction)**.** *The progress choice, DE, and Use derive*

$$\frac{\Gamma \vdash E(0) \qquad \Gamma, E(z) \vdash E(Sz)}{\Gamma \vdash E(t)} \tag{EI}$$

for every equation E, with the usual freshness condition.

Proof. Choose an unused witness coordinate u. Progress DI introduces $\mathsf{D}_{u,y}E(y)$ from the given step. DE opens the clause $\langle E(0); y.E(y)\rangle$. Supply its antecedent using the given base proof and use the clause at t. Close DE. □

Lemma D.5 (Cases, equality, and binder congruence)**.** *The case choice derives, for arbitrary native formulas,*

$$\frac{\Gamma \vdash F(0) \qquad \Gamma \vdash F(Sz)}{\Gamma \vdash F(t)} \tag{NC}$$

with the usual freshness condition. Equality reflexivity, symmetry, transitivity, numerical congruence, and uniform matrix interderivability under D are derivable without using the progress choice.

Proof. For cases, use case DI with an unused witness coordinate and matrix $F(y)$, ignoring its retained base in the successor premise. DE opens $\langle F(0); y.F(y)\rangle$; the given zero proof and Use at t finish.

For reflexivity put $r = I(0, t; u.0)$. Use the computation equation $r = t$ as both transport premises with motive $v = t$ to obtain $t = t$. The usual explicit equality motives give symmetry, transitivity, and numerical congruence.

For binder congruence, open the old paired formula at a fresh witness a and introduce the new one by case DI at a. Convert its retained base to the old base, use the opened old clause at the successor, and convert back. Ordinary cut closes the construction, and the reverse conversion gives the converse. No induction is used. □

The primitive progress choice is still restricted to equations. Allowing it at arbitrary compound matrices would, by the argument for (EI), supply induction for arbitrary formulas and change the intended theory.

D.4 The entire ordinary logical interface

Here is a predecessor-free reconstruction of the ordinary logical operations. It makes explicit that the case choice supplies the logical interface independently of progress induction.

Let $C(v, a, b) := I(v, a; u.b)$ with u unused. Computation gives $C(0, a, b) = a$ and $C(Sv, a, b) = b$. Define the gate on fixed finite formula syntax by

$$G_v(s = t) := (C(v, 0, s) = C(v, 0, t)), \qquad G_v(\mathsf{D}_{x,y}B) := \mathsf{D}_{x,y}G_vB,$$

renaming bound variables away from the control. Equality, binder congruence, and external structural induction give

$$\vdash G_0A, \qquad G_{Sv}A \dashv\vdash A, \qquad A \dashv\vdash \mathsf{D}_{u,y}G_yA \quad (u, y \notin \mathrm{FV}(A)). \tag{Gate and padding}$$

For padding, case DI proves the forward direction. DE, the zero-gate theorem, and Use at 1 prove the reverse direction.

Define a selector between two arbitrary formulas by

$$M_v(s = t, u = w) := (C(v, s, u) = C(v, t, w))$$

and, with aligned fresh paired variables, by

$$\begin{aligned} M_v(\mathsf{D}_{x,y}A, \mathsf{D}_{x,y}B) &:= \mathsf{D}_{x,y}M_v(A, B), \\ M_v(E, \mathsf{D}_{x,y}B) &:= \mathsf{D}_{x,y}M_v(G_yE, B), \\ M_v(\mathsf{D}_{x,y}A, E) &:= \mathsf{D}_{x,y}M_v(A, G_yE), \end{aligned}$$

where E is an equation. Every call decreases maximum input binder depth. Padding and congruence prove $M_0(A, B) \dashv\vdash A$ and $M_{Sv}(A, B) \dashv\vdash B$.

Put $\delta(v) = C(v, 1, 0)$ and $p(z) = I(z, 0; u.\delta(u))$. Only the fixed computations $p(1) = 1$ and $p(2) = 0$ are needed. The full logical macro inventory is

$$\begin{aligned} A \Rightarrow_* B &:= \mathsf{D}_{u,z}M_z(A, B), \\ A \wedge_* B &:= \mathsf{D}_{u,z}G_zM_{p(z)}(A, B), \\ \exists_* x\, A(x) &:= \mathsf{D}_{x,y}G_yA(x), \\ A \vee_* B &:= \exists_* x\, M_x(A, B), \\ \forall_* x\, A(x) &:= A(0) \wedge_* \mathsf{D}_{u,z}G_zA(z), \\ \bot_* &:= (0 = 1). \end{aligned} \tag{Logical macros}$$

Newly bound variables are fresh; u is unused.

Implication introduction obtains its assumption from the zero selector and supplies the consequent at successors, then uses case DI. Elimination opens its clause and uses it at 1. Existential introduction uses its witness and the successor gate; elimination opens at a fresh witness, supplies the zero-gate theorem, and uses the clause at 1.

Disjunction introduction uses witnesses 0 and 1. For elimination, discharge the two branch proofs as $A \Rightarrow_* F$ and $B \Rightarrow_* F$. NC applied to $M_z(A, B) \Rightarrow_* F$ gives this implication at every control: the two endpoint cases follow from the branch implications. Existential elimination provides $M_a(A, B)$ and the implication gives F.

Given A and B, selector endpoints and NC give $M_v(A, B)$ at an arbitrary control. Substitute $v = p(Sz)$ and use the successor gate for conjunction introduction. For projections, open its zero-gated clause and specialize at 1 and 2; the two fixed computations give B and A, respectively.

These rules prove existential numerical coverage:

$$(x = 0) \vee_* \exists_* v\, (x = Sv). \tag{Coverage}$$

Use NC. Zero takes the left injection of reflexivity, and a successor takes the right injection with the evident witness. Coverage gives equality cases even when the retained context contains x; its elimination adds $x = 0$ or $x = Sv$ rather than silently changing that retained context.

Universal introduction follows from a fresh-parameter proof by substituting zero and successors and forming the two conjuncts. For elimination, take its $A(0)$ conjunct and open the paired conjunct. Its base is a zero-gate theorem. Coverage of the target chooses the zero instance from $A(0)$ or a successor instance from the opened clause and gate removal.

Finally, under $0 = 1$, computation and transport give $s = C(0, s, t) = C(1, s, t) = t$ for arbitrary numerical terms. Every equation follows, and external induction on formula syntax with case DI gives every native formula. This supplies falsity elimination. No excluded middle for quantified formulas has entered the construction.

D.5 Whole-domain realizations and both formula composites

For a standard first-order reading, let T_{eq} have intuitionistic equality logic, the same numerical signature and computation equations, numerical coverage, separation $(0 = 1) \to \bot$, and contextual equation induction. Its interpretation of the native syntax is

$$(s = t)^\circ = (s = t), \qquad (\mathsf{D}_{x,y} B)^\circ = \exists x \big(B^\circ(x, 0) \to \forall y B^\circ(x, y)\big),$$

$$\langle A; y.B \rangle^\circ = A^\circ \to \forall y B^\circ.$$

The domain is the entire numerical domain and interpreted equality is actual equality.

Case DI is sound in this realization by numerical coverage. Progress DI is sound by contextual equation induction under the conditional base. Use, DE, transport, and computation have their displayed ordinary meanings. Conversely, the native macros derive every ordinary logical rule, (EI) derives equation induction, and (Coverage) derives coverage. Separation translates to the identity implication $(0 = 1) \Rightarrow_* (0 = 1)$.

For the essential paired formula composite,

$$\mathsf{D}_{x,y} B \dashv\vdash \exists_* x \big(B(x, 0) \Rightarrow_* \forall_* y\, B(x, y)\big). \tag{Paired composite}$$

Forwards, DE opens its clause at a fresh a; under $B(a, 0)$ that clause supplies $B(a, y)$ at a fresh arbitrary y. Introduce the universal, implication, and existential macros and close DE. Backwards, eliminate

the existential at a fresh a. Introduce the paired formula with witness a by case DI: the retained base, supplied implication, and universal elimination at Sz give its successor premise.

The ordinary interpretations of the six logical macros have their ordinary round trips. For conjunction, its true base and the instances $1, 2$ recover both operands; the converse uses selector cases. The universal has all successor instances in its paired conjunct and zero in its first conjunct; coverage gives exactly the ordinary universal. Implication, disjunction, and existential quantification use their selector or gate endpoints. Falsity uses the explicit ordinary separation bridge. External induction on finite formulas, together with (Paired composite), gives both formula composites. Clauses correspond in both directions to $A \Rightarrow_* \forall_* y B$ by the derived logical rules.

The numerical composites are literal identities with internally provable graph $x = y$. No standard-numeral restriction, retract, quotient, or equality reflection assumption is being inserted at this stage. The separately supplied arithmetic bridge identifies T_{eq} with full iPRA; the present change preserves that bridge unchanged because Theorem D.2 preserves every native sequent.

D.6 What the theorem does and does not claim

The new inference interface retains one ordinary assumption at every paired introduction. It preserves the entire consequence relation, not just equations or closed theorems. Consequently the established full-iPRA interpretations and equiconsistency transfer unchanged.

This result is different from retaining only progress introduction and strengthening closure elimination. Here the original elimination rule is retained, and the case and progress introductions coexist. It is also different from allowing arbitrary compound progress, which would supply stronger induction.

A complete-checker comparison must still count the certificate choice, its equation-root restriction, substitution, context handling, computation, and the public input/output interface. The mathematical theorem alone is not a byte-count or minimality theorem. The inference-node bound is distinct from source-size measurements and from serialized certificate size. The proof is a written universal proof; finite regression tests do not constitute a Lean/Rocq formalization of it.

E Arithmetic strength and both ordinary interpretation composites

We spell out the arithmetic bridge to avoid identifying standard numerical agreement with bi-interpretability. First elaborate every fixed iterator into an ordinary function symbol. At stage zero the symbols are $0, S$. Given a fixed earlier-stage term $h(u, \vec{p})$, introduce $i_h(n, a, \vec{p})$ with the two equations

$$i_h(0, a, \vec{p}) = a, \qquad i_h(Sn, a, \vec{p}) = h(i_h(n, a, \vec{p}), \vec{p}).$$

Only finite descriptions occur; the domain contains no code for a universal evaluator. Formal parameters are distinct when a description is declared, while actual arguments can coincide.

Let T_I be ordinary intuitionistic first-order logic with actual equality and genuine falsity, these staged equations, contextual induction for literal equations, and

$$\forall x\,(x = 0 \vee \exists y\, x = Sy), \qquad (0 = S0) \to \bot.$$

The second sentence is the falsity bridge. It must be stated: positive numerical equations by themselves admit a singleton structure. Native explosion from $0 = S0$ and a translation of ordinary falsity to that equation do not authorize silently dropping the bridge from T_I.

The native-to-ordinary translation is

$$(s = t)^\circ = (s = t), \qquad (\mathsf{D}_{x,y} B)^\circ = \exists x\big(B^\circ(x, 0) \to \forall y B^\circ(x, y)\big).$$

A clause $\langle A; y.B\rangle$ becomes $A^\circ \to \forall y B^\circ$. For the auxiliary closure used by the compact checkers, $(\mathsf{Q}_y B)^\circ = B^\circ(0) \to \forall y B^\circ$. The interpretation domain is all elements, defined by $x = x$; equality is actual equality.

Case DI is sound because numerical coverage combines its retained base with its successor proof. Progress DI is sound by literal-equation induction under a temporarily assumed base. Use, DE and transport are ordinary logical inferences under the translations. Conversely the derived logical operations, equation induction, cases and explosion in Section D recover the rules and axioms of T_I. An auxiliary closure is interderivable with $\mathsf{D}_{v,y}B(y)$ for unused v: in one direction use case DI with the closure and its base; in the other eliminate the paired formula and return its supplied closure. Thus formulas and context clauses have the stated meanings in both presentations.

E.1 The arithmetic dependency order

The scalar update has no recursion-counter argument. We do not conceal one in an encoding. The inherited internal proofs, reproduced in the appendices, establish the required arithmetic in this order:

1. Using equation induction and numerical cases, construct a fixed scalar predecessor term and prove $P0 = 0$ and $P(Sn) = n$. Existential coverage then gives $x = 0 \vee x = S(Px)$: in the successor case $x = Sy$, apply P to obtain $Px = y$ and substitute.
2. Construct truncated subtraction and a numerical characteristic of actual equality. Prove its reflection on the entire original domain.
3. Compile each fixed quantifier-free formula into its characteristic equation and recover every contextual quantifier-free induction instance.
4. With that induction available, construct pairing and projections by a bounded square-root search and compile every fixed general primitive-recursive description into a scalar iterator term.

No pairing or faithful equality detector is assumed to obtain predecessor. No appeal to standard-model correctness replaces an internal equality. Every induction fixes its passive parameters; diagonal substitution occurs only into a previously proved identity.

For a target recursion

$$f(0,\vec{p}) = g(\vec{p}), \qquad f(Sn,\vec{p}) = h(n, f(n,\vec{p}),\vec{p}),$$

let π, L, H be the proved pairing and projections. The compiler uses

$$V(n,\vec{p}) = I\big(n, \pi(0, g^*(\vec{p})); u.\pi(SL(u), h^*(L(u), H(u),\vec{p}))\big), \qquad f^*(n,\vec{p}) = H(V(n,\vec{p})).$$

Equation induction proves $L(V(n,\vec{p})) = n$; the two projection identities then prove the target defining equations. The construction is external recursion on each finite PR description, not an internal quantification over arbitrary functions.

Theorem E.1 (Whole-domain bi-interpretability and equiconsistency)**.** *The ordinary realization T_I and full first-order iPRA are bi-interpretable. In both theories $x = y$ defines the isomorphism from the original domain to its double interpretation. The native Choice-DI calculus is therefore a faithful proof presentation of a theory bi-interpretable and equiconsistent with iPRA.*

Proof. Interpret an iterator in iPRA by primitive recursion that ignores its counter argument in the update. iPRA proves its equations, literal equation induction, coverage and separation. In the reverse direction use the scalar compiler. The predecessor and equality-reflection proofs supply the successor axioms and every quantifier-free induction instance. All ordinary logical rules are preserved.

For two iterators with the same seed and provably equal updates, equation induction on the equality of their values proves uniqueness at every counter, with all parameters fixed. The corresponding argument applies to ordinary primitive recursion. Induct externally on a finite description: its double translation satisfies its original defining equations, so internal uniqueness identifies the two functions. This also proves coherence when substitution changes the staged syntactic description of an update.

The domain and equality are unchanged; hence the identity graph is internally total, bijective and preserves every interpreted operation. The displayed ordinary meanings of the logical macros and the paired binder give both formula composites by external induction on finite formula syntax. Clauses are similarly equivalent to their conditional universal meanings. The explicit falsity bridge preserves contradiction in both directions. Effective translations of finite proofs give equiconsistency, without a claim that either theory proves its own consistency. □

F Internal scalar predecessor bootstrap

This scalar bootstrap is inherited from the supplied scalar development [14] and is reproduced to keep the scalar checker self-contained. It is not a new contribution of this paper. The proof uses a factor map fixing odd states and a saturated-iteration construction of predecessor.

The available arithmetic terms are 0, S, and

$$I(n, a; u.h), \qquad I(0, a; u.h) = a, \qquad I(Sn, a; u.h) = h[I(n, a; u.h)/u].$$

The update binds only u. All other variables in it are fixed numerical parameters. We use equality reasoning, the already reconstructed zero/successor cases, and induction on a single literal equation. Every induction below keeps its other parameters fixed. Predecessor, successor injectivity, pairing, and induction on arbitrary quantifier-free formulas are not assumptions of the argument.

F.1 Elementary terms and two finite cycles

Write $1 = S0$ and use larger numerals as successor abbreviations. Define

$$C(n, a, b) = I(n, a; u.b), \qquad Z(n) = C(n, 1, 0), \qquad a + b = I(b, a; u.Su).$$

The update variable of C is absent from its body. Consequently

$$C(0, a, b) = a, \qquad C(Sn, a, b) = b.$$

Numerical cases show that $Z(n)$ is 0 or 1. The usual commutative-monoid equations for addition follow by literal equation induction. For example, $0 + b = b$ and $Sa + b = S(a + b)$ are proved by induction on b; these yield commutativity by a further equation induction, and associativity follows by induction on the last summand. We use these equations to reorder finite sums. The notation $2a$ means $a + a$ and $3a$ means $a + a + a$; no multiplication operation is needed for this bootstrap.

Put

$$N(n) = I(n, 0; u.Z(u)), \qquad j(u) = Z(u) + 2N(u), \qquad M(n) = I(n, 0; u.j(u)).$$

The range of N is contained in $\{0, 1\}$: its zero value is 0, and its successor value is a value of Z. This uses numerical cases on its counter, not induction on a disjunction. Hence

$$N(n) + Z(N(n)) = 1, \qquad Z(Z(N(n))) = N(n), \qquad N(n + 2) = N(n). \tag{1}$$

Here the last equation follows from two successor computations and the preceding equation. Literal-equation inductions give

$$N(2n) = 0, \qquad N(2n+1) = 1, \qquad N(3n) = N(n). \tag{2}$$

For the first equation the successor step adds two to the argument of N; the second follows by one computation. In the third, increasing n adds three to the left argument and therefore applies Z once to both sides of the induction equation.

Finite computations now give

$$j(0) = 1, \qquad j(1) = 2, \qquad j(2) = 0.$$

The proof below needs just these three values and $M(Sn) = j(M(n))$; it does not assume a division algorithm or a theorem characterizing M as remainder modulo three.

F.2 A ramp with a global scaling identity

Define

$$G(n) = I(n, 0; u.u + M(u) + 1).$$

The central invariant is

$$M(G(n)) = N(n). \tag{3}$$

Its zero case is immediate. For its successor step, retain the single induction equation and split the Boolean value $N(n)$. If it is 0, the next G state is $G(n) + 1$; its M value is $j(0) = 1$, equal to $N(Sn)$. If it is 1, the next G state is $G(n) + 2$; two computations of M change its value from 1 to 2 and then to 0, again equal to $N(Sn)$. This proves the invariant by literal-equation induction.

It follows that

$$G(Sn) = G(n) + N(n) + 1, \qquad G(n+2) = G(n) + 3.$$

Equation induction therefore yields

$$G(2n) = 3n. \tag{4}$$

A separate literal-equation induction proves

$$G(3n) = 3G(n) + N(n). \tag{5}$$

For completeness, its successor calculation is

$$\begin{aligned} G(3Sn) &= G(3n) + N(3n) + 4 = 3G(n) + 2N(n) + 4, \\ 3G(Sn) + N(Sn) &= 3G(n) + 3N(n) + 3 + Z(N(n)). \end{aligned}$$

The expressions agree in both Boolean cases for $N(n)$. The base is $0 = 0$. In particular, (5) was proved internally; it was not inferred from an intended floor-function interpretation of G.

Set

$$H(u) = C(N(u), G(u), u).$$

The crucial choice is to retain u when its parity is 1. The preceding equations give

$$H(2u) = 3u, \qquad H(1) = 1, \qquad H(3u) = 3H(u). \tag{6}$$

To check the last identity, use $N(3u) = N(u)$ and split the Boolean $N(u)$. At 0, both sides are $3G(u)$ by (5); at 1, both are $3u$. This argument does not require that an arbitrary number has already been represented as $2v$ or $2v + 1$.

F.3 A comparator from scalar iteration

For every fixed update h, equation induction on n, keeping m, a and all parameters fixed, proves the semigroup identity

$$I(m+n,a;h) = I(n, I(m,a;h);h).$$

The base is the zero computation and the step applies h to the induction equation. This identity will also be used in the later pairing construction.

For a fixed update h, equation induction on y, with its initializer a and all parameters of h fixed, proves the rotation identity

$$I(Sy,a;h) = I(y,h(a);h). \tag{7}$$

Both sides have base $h(a)$, and their successor equations apply the same h to the previous equality. Here and below $I(y,a;h)$ is an abbreviation for one fixed finite scalar term, not quantification over functions.

By (6), another equation induction gives

$$I(y,3a;H) = 3I(y,a;H). \tag{8}$$

Its base is $3a = 3a$. In the step the fixed-parameter induction equation rewrites the new left side as

$$H(3I(y,a;H)) = 3H(I(y,a;H)).$$

The initializer parameter stays a throughout this induction. Equation induction also gives $I(y,1;H) = 1$, since $H(1) = 1$.

Define

$$E(x) = I(x,1;u.2u), \qquad L(x,y) = N\big(I(y,E(x);H)\big).$$

Then the following entire recursion specification is provable:

$$L(0,y) = 1, \qquad L(Sx,0) = 0, \qquad L(Sx,Sy) = L(x,y). \tag{9}$$

The first equation uses $E(0) = 1$ and the fixed-point identity for H. The second uses $E(Sx) = 2E(x)$ and $N(2a) = 0$. For the third,

$$\begin{aligned} L(Sx,Sy) &= N\big(I(Sy,2E(x);H)\big) \\ &= N\big(I(y,H(2E(x));H)\big) \\ &= N\big(I(y,3E(x);H)\big) \\ &= N\big(3I(y,E(x);H)\big) \\ &= N\big(I(y,E(x);H)\big) = L(x,y). \end{aligned}$$

The successive steps use rotation, $H(2a) = 3a$, iterator scaling, and $N(3a) = N(a)$. Thus L has the comparison equations at all arguments internally. Its standard-model meaning as the indicator of $x \le y$ is motivation, not a premise of the following argument.

F.4 Saturated iteration gives predecessor

We now need only the three equations (9). Define

$$B_m(u) = C(L(Su,m),u,Su), \qquad W(t,m) = I(t,0;u.B_m(u)).$$

The subscript in B_m records a fixed parameter; B_m is an open term abbreviation, not a function variable.

Numerical cases on e prove the selector equation

$$C(e, Sa, Sb) = SC(e, a, b).$$

Equivalently one may use equation induction on e, whose induction hypothesis is unused. Therefore

$$\begin{aligned} B_{Sm}(Su) &= C(L(SSu, Sm), Su, SSu) \\ &= C(L(Su, m), Su, SSu) = SB_m(u), \qquad (10) \\ B_{Sm}(0) &= C(L(S0, Sm), 0, S0) = C(L(0, m), 0, S0) = S0. \qquad (11) \end{aligned}$$

Literal-equation induction on t, with m fixed, proves

$$W(St, Sm) = SW(t, m). \qquad (12)$$

The base is (11). The successor step is

$$\begin{aligned} W(SSt, Sm) &= B_{Sm}(W(St, Sm)) \\ &= B_{Sm}(SW(t, m)) \\ &= SB_m(W(t, m)) = SW(St, m). \end{aligned}$$

Only the second line uses the current induction hypothesis, at the same fixed m.

Two subsequent equation inductions give

$$L(Sn, n) = 0, \qquad W(n, n) = n. \qquad (13)$$

The first uses the comparator recurrence. The second uses (12) in its successor step. The substitution $t = m = n$ is made into that already proved universally parameterized equation; no parameter of an active induction hypothesis is changed. Consequently

$$W(Sn, n) = B_n(n) = C(L(Sn, n), n, Sn) = n. \qquad (14)$$

Finally define the finite scalar term

$$P(m) = I\big(m, 0; u.C(L(SSu, m), u, Su)\big). \qquad (15)$$

Its zero equation is immediate. To justify its successor equation without assuming a separate extensionality rule for iterator bodies, put

$$V(t, m) = I\big(t, 0; u.C(L(SSu, Sm), u, Su)\big).$$

The comparator recurrence identifies this update pointwise with $B_m(u)$. Equation induction on t, with m fixed, gives $V(t, m) = W(t, m)$: their base values are 0, and the successor step uses the pointwise update equation and then the induction equation. Now (14) yields

$$\boxed{P(0) = 0, \qquad P(Sm) = V(Sm, m) = W(Sm, m) = m.}$$

F.5 Dependency boundary

All terms in this section are built from $0, S, I$ by finite substitution. All inductions have a single equation as their invariant. Numerical cases and equality transport are the already reconstructed logical resources. Successor injectivity may now be obtained by applying P to an assumed equation $Sx = Sy$. Equality tests, reflection of quantifier-free formulas, pairing, and the recovery of general primitive recursion can then be developed in the established order. They were not used in order to manufacture predecessor.

This argument is a written internal derivation in the stated theory. The finite implementation checks do not constitute a proof-assistant formalization of this entire metatheorem.

G From literal-equation induction to full intuitionistic PRA

This section supplies an unconditional completion theorem. It is adapted with attribution from the current forward-left draft [15]. In the scalar application used here, every function occurring in the proof is already a scalar term: predecessor is supplied by Appendix F, the selector is $C(n,a,b) = I(n,a;u.b)$, addition is $x+y = I(y,x;u.Su)$, and truncated subtraction is $d(x,y) = I(y,x;u.Pu)$. All other functions below are finite compositions of these. Thus this proof does not presuppose that arbitrary primitive recursion has already been compiled. The QFI conclusion first applies to the scalar language; the general PR compiler comes afterward. Literal-equation induction, together with the primitive-recursive defining equations, predecessor coverage, and separation of 0 and 1, proves induction for every quantifier-free formula. In particular, the proof does not assume a faithful numerical equality test, stability or decidability of equality, successor uniqueness, or any commutation law for an iterator. We construct the equality test and prove its reflection property for arbitrary elements of the numerical domain.

G.1 The exact hypotheses

Work in ordinary intuitionistic first-order logic with equality and genuine falsity $\bot$. The language has one numerical sort and a symbol for each fixed, finite primitive-recursive description. Distinct descriptions are not identified merely because they compute the same standard function. Let PRE contain the defining equations for zero, successor, projections, composition, and primitive recursion. In particular, choose descriptions P and C with equations

$$P(0) = 0, \qquad P(Sx) = x, \tag{16}$$

$$C(0,a,b) = a, \qquad C(Sx,a,b) = b. \tag{17}$$

Write $1 = S0$. In addition to PRE, assume

$$\mathsf{Cov}: \quad \forall x\,(x = 0 \vee x = S(Px)), \tag{18}$$

$$\mathsf{Sep}: \quad 0 = 1 \longrightarrow \bot, \tag{19}$$

and induction for each *literal equation* $E(n,\vec{p})$:

$$\forall \vec{p}\,\left[(E(0,\vec{p}) \wedge \forall n(E(n,\vec{p}) \longrightarrow E(Sn,\vec{p}))) \longrightarrow \forall n\, E(n,\vec{p})\right]. \tag{20}$$

Here the schematic expression E is a single equality of numerical terms; the displayed universal quantifiers range only over numbers. Every description and every instance of the schema is finite external syntax.

The axiom formulation (20) and the usual contextual induction rule are interchangeable. For example, if

$$\Gamma \vdash E(0,\vec{p}), \qquad \Gamma, E(z,\vec{p}) \vdash E(Sz,\vec{p}),$$

where z is absent from Γ and the passive parameters, discharge the second premise's equation and universally introduce z. The axiom then gives $\Gamma \vdash \forall n E(n,\vec{p})$, which can be instantiated at any target term. Conversely, assume the antecedent of the axiom and apply the contextual rule with fresh variables. This uses the ordinary intuitionistic logical rules and permits arbitrary, including quantified, formulas in Γ.

G.2 Constructor separation and two elementary tests

Define the sign description and the countdown description by

$$\mathrm{sg}(u) = C(u,0,1), \qquad K(u) = C(u,1,Pu). \tag{21}$$

Their defining equations imply

$$\mathrm{sg}(0) = 0, \qquad \mathrm{sg}(Sx) = 1, \tag{22}$$
$$K(0) = 1, \qquad K(Sx) = x. \tag{23}$$

Constructor laws. Successor is injective: applying P to $Sx = Sy$ gives $x = y$ by (16). Successor never equals zero. Indeed, $Sx = 0$ gives $1 = 0$ by applying sg and using (22); symmetry and (19) give $\bot$. Consequently $1 \neq 0$, where $A \neq B$ abbreviates $A = B \to \bot$. None of these arguments uses induction.

The sign test. For every u,

$$\mathrm{sg}(u) = 0 \vee \mathrm{sg}(u) = 1, \tag{24}$$
$$\mathrm{sg}(u) = 0 \quad \longleftrightarrow \quad u = 0, \tag{25}$$
$$\mathrm{sg}(u) = 1 \quad \longleftrightarrow \quad u \neq 0. \tag{26}$$

To prove the range assertion, split (18). The branch $u = 0$ gives sign 0, and the branch $u = S(Pu)$ gives sign 1. The implication $u = 0 \to \mathrm{sg}(u) = 0$ is computation. For its converse, the successor branch of coverage would make $\mathrm{sg}(u) = 0$ imply $1 = 0$, a contradiction; in that branch explosion gives $u = 0$ as well. Similarly, if $\mathrm{sg}(u) = 1$ and $u = 0$, computation gives $0 = 1$. Conversely, $u \neq 0$ excludes the zero branch of coverage, and the successor branch computes the sign as 1.

Coverage and (17) also give, for all u, v,

$$C(u, v, v) = v. \tag{27}$$

Both coverage branches compute to v, so this identity again needs no induction.

The countdown test and the notation W. For every u,

$$K(u) = 0 \quad \longleftrightarrow \quad u = 1. \tag{28}$$

If $u = 1 = S0$, (23) gives $K(u) = 0$. Conversely, assume $K(u) = 0$ and split coverage. If $u = 0$, then $K(u) = 1$, contradicting separation, and explosion gives $u = 1$. If $u = S(Pu)$, computation gives $K(u) = Pu$; hence $Pu = 0$ and $u = S0 = 1$.

If a presentation uses a description named W with defining equations

$$W(0) = 1, \qquad W(Sx) = x,$$

then it proves $K(u) = W(u)$ for every u already from coverage. In the zero branch both sides are 1; in the branch $u = S(Pu)$ both sides are Pu. Thus K and W are provably equal on the whole domain, even if they are distinct descriptions. This equality does not appeal to standard-model agreement or to uniqueness of an arbitrary recurrence.

G.3 Four elementary uses of equation induction

Choose descriptions for addition and truncated subtraction, with recursion on the second argument:

$$x + 0 = x, \qquad x + Sy = S(x + y), \tag{29}$$
$$d(x, 0) = x, \qquad d(x, Sy) = P(d(x, y)). \tag{30}$$

Using these equations and (20), we now derive

$$0 + y = y, \tag{31}$$
$$(Sx) + y = S(x + y), \tag{32}$$
$$d(Sx, Sy) = d(x, y), \tag{33}$$
$$d(Sa, a) = 1. \tag{34}$$

Each induction below has a single equation as its induction formula.

For (31), induct on y. The base is $0 + 0 = 0$. The step is $0 + Sy = S(0 + y) = Sy$.

For (32), hold x fixed and induct on y. At zero both sides are Sx. At a successor the induction hypothesis gives

$$(Sx) + Sy = S\big((Sx) + y\big) = S\big(S(x + y)\big) = S(x + Sy).$$

Combining (32) with the defining equation for addition gives the useful rearrangement

$$(Sy) + r = y + Sr. \tag{35}$$

Indeed, both sides are $S(y + r)$.

For (33), hold x fixed and induct on y. The base computes as

$$d(Sx, S0) = P(d(Sx, 0)) = P(Sx) = x = d(x, 0).$$

Applying P to the induction equation gives the step:

$$d(Sx, S(Sy)) = P(d(Sx, Sy)) = P(d(x, y)) = d(x, Sy).$$

Finally, induct on a for (34). The base is $d(S0, 0) = S0 = 1$. At a successor,

$$d(S(Sa), Sa) = d(Sa, a) = 1,$$

using (33) and the induction hypothesis.

G.4 An equation invariant that reflects actual equality

The remaining arithmetic ingredient is the following equation, valid for all x, y:

$$C(\mathrm{sg}(d(x, y)),\ x,\ y + d(x, y)) = x. \tag{36}$$

Its meaning is that a nonzero subtraction residual reconstructs the original argument by addition. The selector expresses this assertion as one literal equation, so equation induction applies directly.

Hold x fixed and induct on y. At $y = 0$, subtraction gives residual x, and (31) gives $0 + x = x$. The desired equation is consequently $C(\mathrm{sg}(x), x, x) = x$, which is (27).

For the step, assume (36) at y and abbreviate

$$r = d(x, y), \qquad r' = Pr = d(x, Sy).$$

These are local term abbreviations, not additional function symbols. Split coverage for r'.

If $r' = 0$, then $\mathrm{sg}(r') = 0$, and the successor instance of (36) reduces immediately to $x = x$.

In the other branch, $r' = S(Pr')$, so constructor separation gives $r' \neq 0$. Split coverage for r. Its zero branch would imply $r' = P0 = 0$, a contradiction, from which the desired step equation follows. In its successor branch,

$$r = S(Pr) = Sr', \qquad \mathrm{sg}(r) = 1.$$

The induction equation therefore computes to $y + r = x$. Meanwhile, (35) gives

$$(Sy) + r' = y + Sr' = y + r = x.$$

Since r' is a successor, $\mathrm{sg}(r') = 1$, so this is exactly the branch selected in the successor instance of (36). This completes the equation induction. All case distinctions came from coverage; no induction on an implication or a disjunction was used.

Reflection at residual 1. We have, for arbitrary a, b,

$$d(Sa, b) = 1 \quad \longleftrightarrow \quad a = b. \tag{37}$$

For the forward direction, substitute $x = Sa$, $y = b$ into (36). If $d(Sa, b) = 1$, the sign is 1 and the selector returns its second branch, giving

$$b + 1 = Sa.$$

The defining equation for addition gives $b + 1 = S(b + 0) = Sb$. Thus $Sb = Sa$, and applying P gives $b = a$, hence $a = b$. Conversely, $a = b$ and (34) give $d(Sa, b) = d(Sa, a) = 1$.

Define the equality characteristic by the fixed description

$$e(a, b) = \mathrm{sg}\big(K(d(Sa, b))\big). \tag{38}$$

The sign, countdown, and residual reflection lemmas give

$$e(a, b) = 0 \quad \longleftrightarrow \quad a = b, \tag{39}$$

$$e(a, b) = 0 \ \vee \ e(a, b) = 1. \tag{40}$$

In detail, $e(a, b) = 0$ is equivalent to $K(d(Sa, b)) = 0$ by (25), which is equivalent to $d(Sa, b) = 1$ by (28), and then to $a = b$ by (37). The range assertion is (24) applied to $K(d(Sa, b))$.

The reflection property concerns the logic's actual equality throughout the domain. It was not assumed as part of a numerical comparison symbol, and its proof did not restrict a or b to standard numerals. It also yields decidable equality. In the $e(a, b) = 0$ branch, (39) gives $a = b$. In the $e(a, b) = 1$ branch, an assumption $a = b$ would give $e(a, b) = 0$ and therefore $0 = 1$, contradicting separation. Hence

$$\forall a \, \forall b \, \big(a = b \vee (a = b \to \bot)\big) \tag{41}$$

is a theorem, not an extra hypothesis.

G.5 Compilation of all quantifier-free formulas

For each fixed quantifier-free formula $A(\vec{p})$, construct a term $b_A(\vec{p})$ by external recursion on its finite syntax. Take equality, $\bot$, $\wedge$, $\vee$, and $\to$ as the primitive formula constructors; negation is implication into $\bot$. Set

$$b_{s=t} = e(s, t), \qquad b_\bot = 1, \tag{42}$$

$$b_{A \wedge D} = C(b_A, b_D, 1), \tag{43}$$

$$b_{A \vee D} = C(b_A, 0, b_D), \tag{44}$$

$$b_{A \to D} = C(b_A, b_D, 0). \tag{45}$$

Every term here is a finite composition of fixed primitive-recursive descriptions; if desired, projections and composition give it a single description symbol. No internal quantification over formulas or programs is involved.

An external induction on the syntax of A proves, within the arithmetic theory,

$$(b_A = 0 \lor b_A = 1) \quad \text{and} \quad (b_A = 0 \longleftrightarrow A). \tag{46}$$

Here are the full connective checks. In every case $b_A = 1$ implies $\neg A$: if A held, the already established implication $A \to b_A = 0$ would contradict separation.

Atomic formulas and falsity. For an equation, both claims are (39)–(40). For falsity the range assertion is $1 = 0 \lor 1 = 1$, with its right disjunct proved by reflexivity. Also $1 = 0 \to \bot$ follows from separation and symmetry, while $\bot \to 1 = 0$ follows by explosion. Thus $1 = 0$ is equivalent to $\bot$, as required.

Conjunction. Split $b_A = 0 \lor b_A = 1$. In the zero branch, A holds and $b_{A\land D} = b_D$. Its range follows from the range of b_D, and its being zero is equivalent to D, hence to $A \land D$ under the known A. In the one branch, $\neg A$ holds and $b_{A\land D} = 1$. Its range is immediate. If it were zero, separation would give $\bot$, hence $A \land D$; conversely an assumption $A \land D$ contradicts $\neg A$, and explosion gives $b_{A\land D} = 0$.

Disjunction. In the branch $b_A = 0$, the formula A holds and $b_{A\lor D} = 0$. The range is immediate, and both $b_{A\lor D} = 0$ and $A \lor D$ hold, proving their equivalence. In the branch $b_A = 1$, we have $\neg A$ and $b_{A\lor D} = b_D$. Its range follows from the range of b_D. If its value is zero, then D and hence $A \lor D$. Conversely, from $A \lor D$, the A alternative contradicts $\neg A$ and yields $b_D = 0$ by explosion, while the D alternative yields $b_D = 0$ directly. This proves the required equivalence in both alternatives.

Implication. In the branch $b_A = 0$, the formula A holds and $b_{A\to D} = b_D$. Its range follows from the range of b_D. If its value is zero, then D, which gives $A \to D$. Conversely, $A \to D$ and the known A give D, hence $b_D = 0$. In the branch $b_A = 1$, we have $\neg A$ and $b_{A\to D} = 0$. Its range is immediate. To prove $A \to D$, assume A; this contradicts $\neg A$, and explosion gives D. Thus both $b_{A\to D} = 0$ and $A \to D$ hold in this branch, and their equivalence follows. This argument uses the proved two-value range of the finite characteristic. It does not use excluded middle for arbitrary formulas or change the surrounding intuitionistic logic.

This completes the external syntactic induction proving (46) for every quantifier-free formula.

G.6 Recovery of induction and the full theory

Let $A(n, \vec{p})$ be any fixed quantifier-free formula. Suppose an arbitrary retained context Γ proves its base and its induction step:

$$\Gamma \vdash A(0, \vec{p}), \qquad \Gamma, A(z, \vec{p}) \vdash A(Sz, \vec{p}),$$

with the usual freshness condition on z. By (46), the base gives $\Gamma \vdash b_A(0, \vec{p}) = 0$. Under the additional assumption $b_A(z, \vec{p}) = 0$, the same equivalence gives $A(z, \vec{p})$, the given step gives $A(Sz, \vec{p})$, and the equivalence gives $b_A(Sz, \vec{p}) = 0$. Apply literal-equation induction to $b_A(n, \vec{p}) = 0$ and then apply its reflection implication. The result is $\Gamma \vdash A(t, \vec{p})$ for every target term t. Thus the theory proves the full contextual quantifier-free induction rule, including in arbitrary quantified contexts.

Completion theorem. In the common language of fixed primitive-recursive descriptions, the theory

$$\mathsf{PRE} + \mathsf{Cov} + \mathsf{Sep} + \mathsf{EI}$$

has exactly the same sequents as ordinary first-order intuitionistic primitive recursive arithmetic with all quantifier-free induction instances. The forward simulation follows from the recovered quantifier-free induction, successor injectivity, and successor separation; all primitive-recursive defining equations and all logical rules are shared. For the reverse simulation, literal-equation induction is an instance of quantifier-free induction, and successor separation gives Sep. Quantifier-free induction proves Cov: at zero choose its left disjunct, and at a successor use $Sx = S(P(Sx))$, which follows from $P(Sx) = x$, for its right disjunct.

Both interpretations therefore use the full domain, defined by $x = x$, and preserve every function symbol, equality, logical connective, and quantifier. Both composites have the identity isomorphism, with graph $x = y$. The theories are consequently identity-domain bi-interpretable and equiconsistent. This completion theorem is unconditional once the literal-equation induction rule has been obtained; it has no dependency on a classical successor-uniqueness reduction.

H Pairing, primitive recursion, and internal numerical composites

This appendix is adapted with attribution from the progress-closure arithmetic development [16]. Its prerequisites have now been proved: scalar predecessor, arithmetic reflection, and full quantifier-free induction. Pairing is a derived proof tool; it is not part of any checker.

H.1 Pairing from a bounded square-root search

Once QFI is available, a familiar square-based injection gives a particularly direct compiler. Write $a \le b$ for $d(a, b) = 0$. The usual linear-order and semiring identities follow by fixed quantifier-free inductions from the successor, addition, and monus equations. Multiplication itself is the scalar term

$$a \cdot b = I(b, 0; u.u + a).$$

Only primitive-recursive arithmetic is involved in the following bounded root search; there is no minimization over an unbounded domain.

For a fixed parameter n, put

$$U_n(v) = C(d((Sv)^2, n), Sv, v), \qquad R_n(i) = I(i, 0; v.U_n(v)), \qquad q(n) = R_n(n).$$

The update increments v exactly when $(v + 1)^2 \le n$, and otherwise retains it. QFI in i, with n fixed, proves

$$R_n(i) \le i, \qquad R_n(i)^2 \le n, \qquad R_n(i) = i \ \vee \ n < (S(R_n(i)))^2. \tag{Root invariant}$$

The base has $R_n(0) = 0$. If the update increments, its test contradicts the strict-upper-bound alternative in the third invariant. Thus the old state equals i, and the new state equals Si; its square bound is the increment test. If the update does not increment, the strict upper square bound holds and the state remains fixed. An already established strict upper square bound forces every subsequent update to remain fixed. These are exactly the cases needed for the three displayed invariants.

At $i = n$, either the strict upper square bound already holds or $R_n(n) = n$, in which case $n < (n+1)^2$ supplies it. Thus

$$q(n)^2 \le n < (q(n) + 1)^2. \tag{Root}$$

Monotonicity of squares shows uniqueness of a number satisfying Root. All formulas just used are quantifier free in the scalar language, with fixed n during the induction.

Now define

$$\pi(a,b) = (a+b)^2 + a, \qquad L(n) = d(n, q(n)^2), \qquad H(n) = d(q(n), L(n)).$$

For $k = a + b$, one has $k^2 \le k^2 + a < (k+1)^2$ since $a \le k$. Root uniqueness gives $q(\pi(a,b)) = a + b$, and therefore

$$L(\pi(a,b)) = a, \qquad H(\pi(a,b)) = b.$$

Surjectivity of π is unnecessary. The original numerical domain is never replaced by the range of this coding; pairing is used only to store the finite state of a fixed primitive recursion. This standard square pairing is a convenient reconstruction here, not a claimed new arithmetic basis.

H.2 Compiling each primitive recursion

Suppose the fixed target description is

$$f(0, \vec{p}) = g(\vec{p}), \qquad f(Sn, \vec{p}) = h(n, f(n, \vec{p}), \vec{p}).$$

Assume by external structural induction that the earlier g, h already have scalar translations g^*, h^*. Define

$$W(u, \vec{p}) = \pi(SL(u), h^*(L(u), H(u), \vec{p})),$$

$$V(n, \vec{p}) = I(n, \pi(0, g^*(\vec{p})); u.W(u, \vec{p})), \qquad f^*(n, \vec{p}) = H(V(n, \vec{p})).$$

The update has only its state variable bound, with every p_i fixed. Equation induction proves $L(V(n, \vec{p})) = n$. The two projection equations and scalar computation then prove both defining equations for f^* internally. Initial functions and composition translate directly. This completes the translation of every fixed finite PR description without adding a universal evaluator.

H.3 Why both numerical round trips are internal

Two fixed scalar iterators with the same seed and provably equal updates are internally equal at all counters: use literal-equation induction comparing their values. Their base agrees, and their successor equations plus congruence prove the step. The same uniqueness argument works for ordinary primitive recursion.

For an ordinary PR description, induct externally on its finite construction. The double translation satisfies its original defining equations, so internal equation induction identifies it with the original operation. For a native scalar description, earlier translated updates agree by the external induction hypothesis; scalar uniqueness identifies the double-translated iterator. These proofs hold at arbitrary internal counters.

Thus the domain relation $x = y$ is provably a total bijection preserving every fixed numerical operation in both composites. The formula constructions and round trips in Section D give the corresponding formula composites. Both interpretations preserve falsity, taking native $0 = 1$ to an ordinary contradiction and ordinary $\bot$ to native $0 = 1$. Finite proof translations therefore establish equiconsistency.

References

[1] Vladimir Reshetnikov, repository maintainer. *ProveIt, current draft collection*, commit `76a2651601d17b34d46753f189ea026a21ce8d29`. https://github.com/VladimirReshetnikov/ProveIt/tree/76a2651601d17b34d46753f189ea026a21ce8d29/docs/drafts.

[2] *Bounded Initializers*, `ipra_bounded_initializer_2026-11-10.zip` in [1]. The prior counter-aware tied-seed B_ϵ basis, actual equality, bounded computation, finite-case graphs, and majorant method.

[3] *A Certified Zero Removes a Computation Opcode*, `ipra_certified_zero_2026-11-10.zip` in [1]. Source of the previous minima and the inherited normalization.

[4] *Local Proof Scopes and Interpretation Obstructions*, `ipra_local_scopes_2026-11-10.zip` in [1].

[5] *An Arity Calculus for iPRA*, `ipra_arity_calculus_2026-11-10.zip` in [1].

[6] *A Smaller Complete iPRA Checker and a Symmetry Obstruction*, `ipra_compact_kernel_2026-11-10.zip` in [1].

[7] *Uniform Predecessor Certificates for Intuitionistic PRA*, `ipra_uniform_predecessor_2026-11-10.zip` in [1].

[8] *Constructed Computation*, `ipra_constructed_computation_2026-11-10.zip` in [1].

[9] *Numerical Axioms as Local Assumptions*, inherited in the reference material of [5, 8].

[10] *A One-State Recursor for iPRA*, `ipra_transient_seed_2026-11-10.zip` in [1].

[11] *Affine Depths for a Small iPRA Checker*, `ipra_affine_arity_2026-11-10.zip` in [1].

[12] Saeed Salehi. *Decidable Formulas of Intuitionistic Primitive Recursive Arithmetic*. Reports on Mathematical Logic **36** (2002), 55–61. https://rml.tcs.uj.edu.pl/rml-36/36-SALEHI.pdf.

[13] Jeremy Avigad. *Proof Mining*, ASL 2004 lecture slides, especially slides 12–13 on primitive recursive arithmetic. https://www.andrew.cmu.edu/user/avigad/Talks/asl04.pdf.

[14] Inherited scalar predecessor development, retained with attribution in `reference/scalar_predecessor.tex` of [5].

[15] Inherited forward-left equality-reflection development, retained in `reference/equality_reflection.tex` of [5].

[16] Inherited progress-closure pairing and primitive-recursion compiler, retained in `reference/primitive_recursion.tex` of [5].